

자연어처리와 컴퓨터 비전 6장-

📌 과제 유형	예습
📅 마감일	@2025년 5월 19일
☑ 발표	☐
☑ 완료	☐
📌 주차	11주차

Word2Vec

- **Word2Vec**은 2013년 구글에서 공개한 임베딩 모델로 단어 간 유사성을 측정하기 위해 **분포 가설(distributional hypothesis)**을 기반으로 개발되었다.
- 분포 가설: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정이다. 분포 가설은 단어 간의 **동시 발생(co-occurence)** 확률 분포를 이용해 단어 간의 유사성을 측정한다.
 - '내일 자동차를 타고 부산에 간다'와 '내일 비행기를 타고 부산에 간다'라는 두 문장에서 '자동차'와 '비행기'는 주변에 분포한 단어들이 동일하거나 유사하므로 비슷한 의미를 가질 것이라고 예상한다.
- 이러한 가정을 통해 **분산 표현(Distributed Representation)**을 학습할 수 있다. 분산 표현이란 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것을 의미한다.
- 분포 가설에 따라 단어의 의미는 **문맥상 분포적 특성**을 통해 나타난다. 즉, 유사한 문맥에서 등장하는 단어는 **비슷한 벡터 공간상 위치**를 갖게 된다. ➡ '자동차'와 '비행기'라는 단어는 벡터 공간에서 서로 가까운 위치에 표현된다.
- 대량의 텍스트 데이터에서 단어 간의 관계를 파악하고 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미를 효과적으로 표현했다. 다양한 자연어 처리 작업에서 높은 성능을 보여주며, 다운 스트림 작업에서 더 뛰어난 성능을 보인다.

단어 벡터화

- 단어를 벡터화하는 방법은 크게 **희소 표현(sparse representation)**과 **밀집 표현(dense representation)**으로 나눌 수 있다.
- **희소 표현**: 원-핫 인코딩, TF-IDF 등의 빈도 기반 방법
 - 대부분의 벡터 요소가 0으로 표현

- 모델의 단어 사전이 5000개의 단어로 이루어졌다면 10개의 토큰으로 이루어진 입력 텍스트를 원-핫 인코딩으로 표현하면 최소한 4990개의 0이 포함된 벡터로 표현됨
- 단어 사전 크기가 커지면 벡터 크기도 커지므로 공간적 낭비가 발생하고, 단어 간의 유사성을 반영하지 못하고 벡터 간 유사성을 계산하는데도 많은 비용이 발생한다.

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

- 밀집 표현: Word2Vec

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814
외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

- 단어를 고정된 크기의 실수 벡터로 표현하기 때문에 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않는다. 벡터 공간 상에서 단어 간의 거리를 효과적으로 계산할 수 있으며, 벡터의 대부분이 0이 아닌 실수로 이뤄져 있어 효율적으로 공간을 활용할 수 있음
- 학습을 통해 단어를 벡터화하기 때문에 단어의 의미를 비교할 수 있다. 밀집 표현된 벡터를 단어 임베딩 벡터(Word Embedding Vector)라고 하며, Word2Vec은 대표적인 단어 임베딩 기법 중 하나이다.
- Word2Vec은 밀집 표현을 위해 CBoW와 Skip-gram 두 가지 방법을 사용한다.

CBow(Continuous Bag of Words)

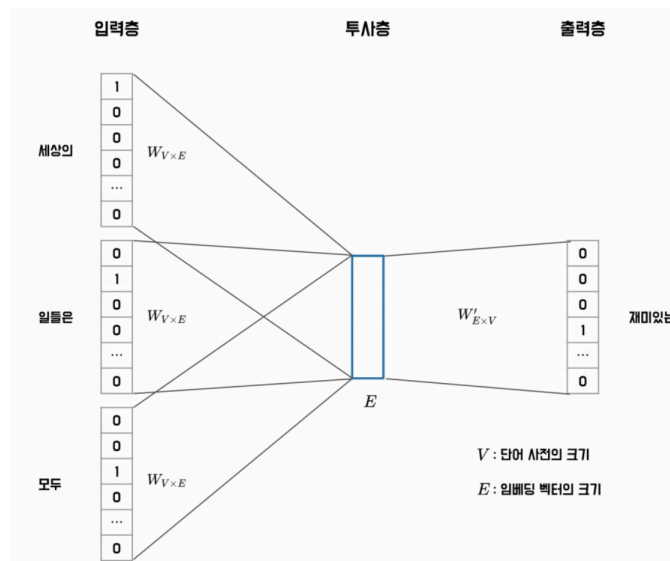
- 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법
- **Center Word(중심 단어)**는 예측해야할 단어, 예측에 사용되는 단어들은 **Context Word(주변 단어)**
- 중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지를 **Window(윈도)**로 설정한다. 이 윈도를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여 마지막 단어까지 학습한다. 양 옆으로 카운트

- 학습을 위해 윈도우를 이동해가며 학습하는데, 이러한 방법을 **Sliding Window**라 한다. CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습할 수 있다.
- 학습 데이터는 **(주변 단어 | 중심 단어)**로 구성된다. 다음은 하나의 입력 문장에서 윈도우 크기가 2일 때 학습 데이터가 어떻게 구성되는지 보여준다.

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

- 대량의 말뭉치에서 효율적으로 단어의 분산 표현을 학습할 수 있으며, 얻어진 학습 데이터는 다음 그림과 같은 구조의 인공 신경망을 학습하는데 사용된다.



- CBoW 모델은 각 입력 단어의 원-핫 벡터를 입력값으로 받는다. 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터를 예측한다.

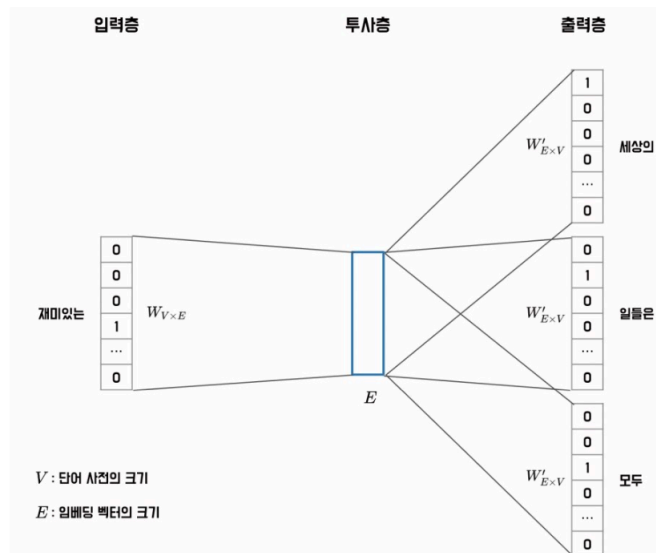
- 입력 단어는 원-핫 벡터로 표현되어 투사층(Projection Layer)에 입력된다. 투사층이란 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표(Lookup table, LUT) 구조가 된다.
- 투사층을 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환된다. 입력된 세 단어의 임베딩 벡터를 $V_{\text{세상의}}$, $V_{\text{일들은}}$, $V_{\text{모두}}$ 라고 가정하면 이 벡터들의 평균값을 계산한다.
- 계산된 벡터를 가중치 행렬 $W'_{E \times V}$ 와 곱하면 V 크기의 벡터를 얻는다. 이 벡터에 소프트 맥스 함수를 이용해 중심 단어를 예측한다.

Skip-Gram

- Skip-Gram은 CBoW와 반대로 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델이다. 따라서 Skip-gram은 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터 세트를 만든다.
- 이때 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델을 학습시킨다. 다음 그림은 Skip-gram에서 윈도우 크기가 2일 때 학습 데이터가 어떻게 구성되는지 보여준다.

입력 문장	학습 데이터
{세상의 재미있는 일들은}모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
{세상의 재미있는 일들은 모두}밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
{세상의 재미있는 일들은 모두 밤에}일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의{재미있는 일들은 모두 밤에}일어난다	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는{일들은 모두 밤에}일어난다	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은{모두 밤에}일어난다	(일어난다 모두), (일어난다 밤에)

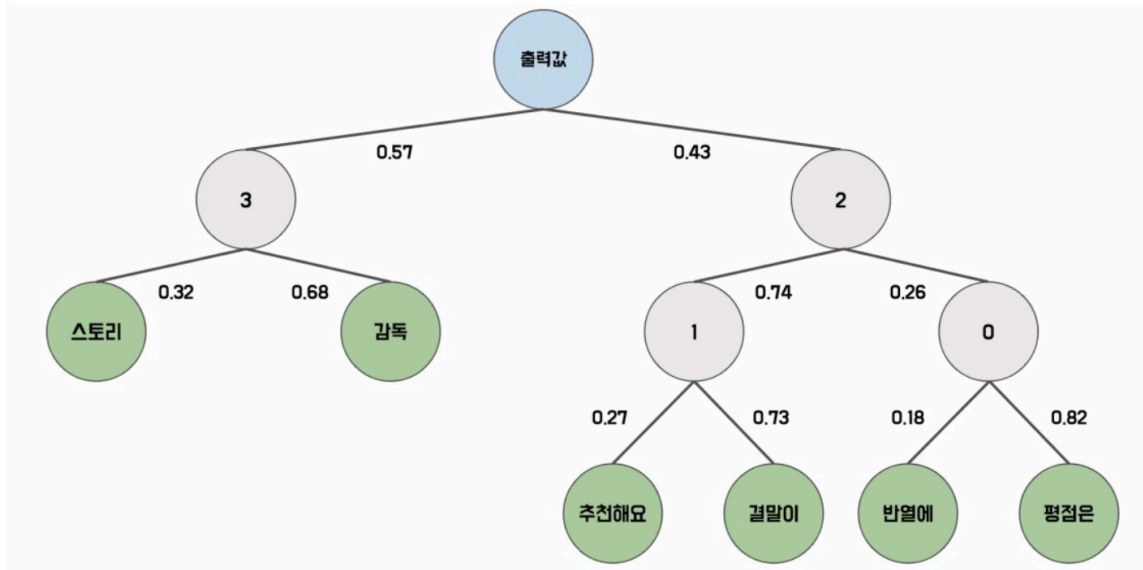
- CBoW는 하나의 윈도우에서 하나의 학습 데이터가 만들어지는 반면, Skip-gram은 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어진다.
- Skip-gram은 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측하므로 더 많은 학습 데이터 세트를 추출할 수 있으며, 일반적으로 CBoW보다 더 뛰어난 성능을 보인다.
- Skip-gram은 비교적 드물게 등장하는 단어를 더 잘 학습할 수 있게 되고 단어 벡터 공간에서 더 유의미한 관계를 형성할 수 있다.



- Skip-gram 모델도 CBoW와 마찬가지로 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져온다. 입력 단어의 임베딩과 $W'_{E \times V}$ 가중치와 곱셈을 통해 V 크기의 벡터를 얻고, 이 벡터에 소프트맥스 연산을 취함으로써 주변 단어를 예측한다.
- 소프트맥스 연산은 모든 단어를 대상으로 내적 연산을 수행한다. 말뭉치의 크기가 커지면 단어 사전 크기도 커지므로 대량의 말뭉치를 통해 Word2Vec 모델을 학습할 때 **학습 속도가 느려지는** 단점이 있다. 이를 해결하기 위해 **계층적 소프트 맥스와 네거티브 샘플링 기법**을 적용해 학습 속도가 느려지는 문제를 완화한다.

계층적 소프트 맥스 (Hierarchical Softmax)

- 출력층을 이진 트리 구조로 표현해 연산을 수행한다. 이때 자주 등장하는 단어일 수록 트리의 상위 노드에 위치하고, 드물게 등장하는 단어일수록 하위 노드에 배치된다.
- 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성을 보인다.



- 각 노드는 학습이 가능한 벡터를 가지며, 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수를 통해 확률을 계산한다.
- **잎 노드(Leaf Node)**는 가장 깊은 노드로, 각 단어를 의미하며, 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 한다. 각 단어의 확률은 경로 노드의 확률을 곱해서 구할 수 있다.
 - “추천해요” 단어는 $0.43 * 0.74 * 0.27 = 0.085914$ 의 확률을 갖게 된다. 이 경우 학습 시 1, 2번 노드의 벡터만 최적화하면 된다.
- 단어 사전의 크기를 V 라고 했을 때 일반적인 소프트맥스 연산은 $O(V)$ 의 시간 복잡도를 갖지만, 계층적 소프트맥스의 시간 복잡도는 $O(\log_2 V)$ 의 시간 복잡도를 갖는다.

네거티브 샘플링 (Negative Sampling)

- Word2Vec 모델에서 사용되는 확률적인 샘플링 기법으로 전체 단어 집합에서 **일부 단어를 샘플링하여 오답 단어로 사용한다.**
- 학습 원도 내에 등장하지 않는 단어를 n (n : 5~20)개 추출하여 정답 단어와 함께 소프트맥스 연산을 수행한다. 이를 통해 전체 단어의 확률을 계산할 필요 없이 모델을 효율적으로 학습할 수 있다.
- 네거티브 샘플링 추출 확률을 계산하기 위해 먼저 각 단어 w_i 의 출현 빈도수 $f(w_i)$ 로 나타낸다. 가령 말뭉치 내에 단어 ‘추천해요’가 100번 등장했고, 전체 단어의 빈도가 2,000이라면 $f(\text{추천해요}) = 100/2000 = 0.05$ 가 된다.
- $P(w_i)$ 는 각 단어가 네거티브 샘플로 추출될 확률이다. 이때 출현 빈도수에 0.75 제곱한 값을 정규화 상수로 사용하는데, 이 값은 실험을 통해 얻어진 최적의 값이다.

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- 네거티브 샘플링에서는 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 아니면 네거티브 샘플링으로 생성된 단어 쌍인지 이진분류를 한다. 이를 위해 **로지스틱 회귀 모델**을 사용하며, 이 모델의 학습 과정에서는 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습한다.
- Skip-gram 모델과 네거티브 샘플링 Word2Vec 모델의 훈련 데이터가 어떻게 다른지 보여준다.

입력 데이터	출력 데이터
재미있는, 세상인	1
재미있는, 일들은	1
재미있는, 모두	1
재미있는, 곁어가다	0
재미있는, 주십시오	0
재미있는, 미리	0
재미있는, 안녕하세요	0

- ‘재미있는’이라는 중심 단어를 통해 세 쌍의 학습 데이터를 추출한다. 네거티브 샘플링 Word2Vec 모델은 실제 데이터에서 추출된 단어 쌍은 1로, 네거티브 샘플링으로 추출된 가짜 단어 쌍은 0으로 레이블링하여 다중 분류에서 이진 분류로 학습 목적을 바꾼다.
- 일반적인 Word2Vec 모델과 네거티브 샘플링 Word2Vec 모델의 차이

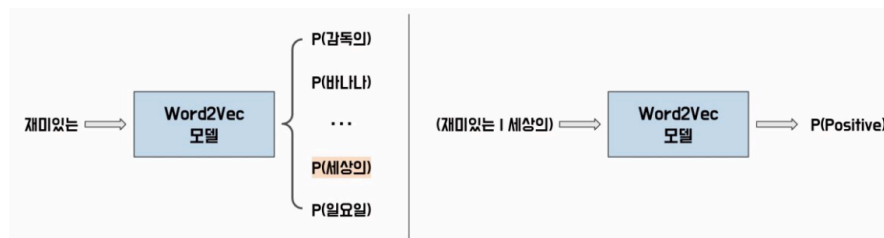


그림 6.10 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

- 네거티브 샘플링 모델에서는 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블을 가져와 내적 연산을 수행한다. 내적 연산을 통해 얻은 값은 시그모이드 함수를 통해 확률값으로 변환된다.

- 이때 레이블이 1인 경우 해당 확률값이 높아지도록, 레이블이 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화된다.

모델 실습: Skip-gram

- Word2Vec 모델은 학습한 단어 수를 V 로, 임베딩 차원을 E 로 설정해 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬을 최적화하여 학습한다. 이때, $W_{V \times E}$ 행렬은 **룩업(Lookup)** 연산을 수행하는데, **임베딩** 클래스를 사용하면 간편하게 구현할 수 있다.
 - 룩업 연산: 배열이나 리스트 등의 데이터 구조에서 인덱스를 이용해 해당 값을 찾아 오는 연산, 이산 변수 값을 연속적인 벡터로 변환하는 과정
 - 임베딩 클래스: 단어나 범주형 변수와 같은 이산 변수를 연속적인 벡터 형태로 변환해 사용할 수 있다. 연속적인 벡터 표현은 모델이 학습하는 동안 단어의 의미와 관련된 정보를 포착하고, 이를 기반으로 단어 간 유사도를 계산한다.

임베딩 클래스

```
embedding = torch.nn.Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx = None,
    max_norm = None,
    norm_type = 2.0
)
```

- **임베딩 수(num_embeddings):** 이산 변수의 개수로 단어 사전의 크기
- **임베딩 차원(embedding_dim):** 임베딩 벡터의 차원수, 임베딩 벡터의 크기
- **패딩 인덱스 (padding_idx):** 패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터를 0으로 설정한다. 병렬 처리를 위해 입력 배치의 문장 길이가 동일해야하므로 입력 문장들을 일정한 길이로 맞추는 역할을 한다. 패딩 인덱스는 임베딩 수보다 작아야하며 패딩 인덱스의 벡터값은 모델 학습시 최적화되지 않는다.
- **노름 타입 (norm_type):** 임베딩 벡터의 크기를 제한하는 방법을 선택한다. 기본값은 2로, 이는 L2 정규화 방식을 사용한다는 의미이다. 만약 노름 타입을 1을 설정하면 L1 정규화 방식을 사용한다.
- **최대 노름 (max_norm):** 임베딩 벡터의 최대 크기를 지정한다. 각 임베딩 벡터의 크기가 최대 노름값 이상이면 임베딩 벡터를 최대 노름 크기로 잘라내고 크기를 감소시킨다.

6.3 기본 Skip-gram 클래스


```

from torch import nn

class VanillaSkipgram(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim= embedding_dim
        )

        self.linear = nn.Linear(
            in_features=embedding_dim,
            out_features=vocab_size
        )

    def forward(self, input_ids):
        embeddings = self.embedding(input_ids)
        output = self.linear(embeddings)
        return output

```

- 계층적 소프트 맥스나 네거티브 샘플링과 같은 효율적인 기법을 사용하지 않는 기본 형식의 Skip-gram 모델은 간단히 구현할 수 있다. 이 모델은 단순히 입력 단어와 주변 단어를 록업 테이블에서 가져와서 내적을 계산한 다음, 손실 함수를 통해 예측 오차를 최소화하는 방식으로 학습된다.
- 기본 형식의 Skip-gram 모델을 선언했다면 모델 학습에 사용할 데이터셋을 불러온다. 데이터셋은 코포라 라이브러리의 네이버 영화 감정 분석 데이터셋을 불러온다.

6.4 영화 리뷰 데이터셋 전처리

```

import pandas as pd
from Korpora import Korpora
from konlpy.tag import Okt

corpus = Korpora.load('nsmc')
corpus = pd.DataFrame(corpus.test)

tokenizer = Okt()

```

```
tokens = [tokenizer.morphs(review) for review in corpus.text]
print(tokens[:3])
```

```
[['굳', 'ㅋ'], ['GDNTOPCLASSINTHECLUB'], ['뭐', '야', '이', '평점', '들', '은', '....', '나쁜진',  
'않지만', '10', '점', '짜', '리', '는', '더', '더욱', '아니잖아']]
```

- 학습은 데이터셋의 크기가 작은 테스트 데이터셋을 활용해 실습을 진행한다.

`corpus.test` 로 테스트 세트를 불러오고 이 데이터셋을 `Okt` 토큰라이저를 활용해 형태소로 추출한다.

6.5 단어 사전 구축

```
from collections import Counter
```

```
def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab
```

```
vocab = build_vocab(corpus = tokens, n_vocab = 5000, special_tokens = ["<u  
token_to_id = {token: idx for idx, token in enumerate(vocab)}  
id_to_token = {idx: token for idx, token in enumerate(vocab)}
```

```
print(vocab[:10])  
print(len(vocab))
```

```
['<unk>', '.', '이', '영화', '의', '...', '가', '에', '...', '을']  
5001
```

- Okt 토큰라이저를 통해 토큰화된 데이터를 활용해 `build_vocab` 함수로 단어 사전을 구축한다.
- `n_vocab` 매개변수는 구축할 단어 사전의 크기이다. 만약 문서 내에 `n_vocab` 보다 많은 종류의 토큰이 있다면, 가장 많이 등장한 토큰 순서로 사전을 구축한다.

- `special_tokens` 는 특별한 의미를 갖는 토큰들을 의미한다. `<unk>` 토큰은 OOV에 대응하기 위한 토큰으로 단어 사전 내에 없는 모든 단어는 `<unk>` 토큰으로 대체된다.
 - 이 예제에서는 단어 사전의 최대 길이를 5000으로 설정했고, 특수 토큰은 1개이므로 단어 사전은 5001개로 구성된다.

6.6 Skip-gram의 단어 쌍 추출

```
def get_word_pairs(tokens, window_size):
    pairs = []
    for sentence in tokens:
        sentence_length = len(sentence)
        for idx, center_word in enumerate(sentence):
            window_start = max(0, idx - window_size)
            window_end = min(sentence_length, idx + window_size + 1)
            center_word = sentence[idx]
            context_words = sentence[window_start: idx] + sentence[idx + 1: window_end]
            for context_word in context_words:
                pairs.append([center_word, context_word])
    return pairs

word_pairs = get_word_pairs(tokens, window_size=2)
print(word_pairs[:5])
```

```
[['굴', 'ㅋ'], ['ㅋ', '굴'], ['뭐', '야'], ['뭐', '이'], ['야', '뭐']]
```

- `get_word_pairs` 함수는 토큰을 입력받아 skip-gram 모델의 입력 데이터로 사용할 수 있게 전처리한다. `window_size` 는 주변 단어를 몇 개 까지 고려할 것인지 설정한다. 각 문장에서는 중심 단어와 주변 단어를 고려하여 쌍을 생성한다.
- `idx` 는 현재 단어의 인덱스를 나타내며, `center_word` 는 중심 단어를 나타낸다. `window_start` 와 `window_end` 는 현재 단어에서 얼마나 멀리 떨어진 주변 단어를 고려할 것인지를 결정한다. `window_start` 와 `window_end` 는 문장 경계를 넘어가지 않도록 조정한다.
- 출력 결과를 보면 각 단어쌍은 [중심 단어, 주변 단어]로 구성되어있다. 임베딩 층은 단어의 **인덱스**를 입력으로 받기 때문에 단어 쌍을 인덱스 쌍으로 변환해야한다.

6.7 인덱스 쌍 변환

```
def get_index_pairs(word_pairs, token_to_id):
    pairs = []
```

```

unk_index = token_to_id("<unk>")
for word_pair in word_pairs:
    center_word, context_word = word_pair
    center_index = token_to_id.get(center_word, unk_index)
    context_index = token_to_id.get(context_word, unk_index)
    pairs.append([center_index, context_index])
return pairs

index_pairs = get_index_pairs(word_pairs, token_to_id)
print(index_pairs[:5])

```

```
[[595, 100], [100, 595], [77, 176], [77, 2], [176, 77]]
```

- `get_index_pairs` 함수는 `get_word_pairs` 함수에서 생성된 단어 쌍을 토큰 인덱스 쌍으로 변환한다. 앞서 생성한 `word_pairs` 와 단어와 단어에 해당하는 ID를 매핑한 딕셔너리인 `token_to_id` 로 인덱스 쌍을 생성한다.
- 딕셔너리의 `get` 메서드로 토큰이 단어 사전 내에 있으면 해당 토큰의 인덱스를 반환하고, 단어 사전 내에 없다면 `<unk>` 토큰의 인덱스를 반환한다.
- 이렇게 생성된 인덱스 쌍은 Skip-gram 모델의 입력 데이터로 사용된다.

6.8 데이터로더 적용

```

import torch
from torch.utils.data import TensorDataset, DataLoader

index_pairs = torch.tensor(index_pairs)
center_indexes = index_pairs[:, 0]
context_indexes = index_pairs[:, 1]

dataset = TensorDataset(center_indexes, context_indexes)
dataloader = DataLoader(dataset, batch_size = 32, shuffle = True)

```

- `index_pairs` 는 `get_index_pairs` 함수에서 생성된 중심 단어와 주변 단어 토큰의 인덱스 쌍으로 이루어진 리스트이다. 이 리스트를 텐서 형식으로 변환하면 $[N, 2]$ 구조를 가지게 된다. 텐서를 중심 단어와 주변 단어로 나눠 데이터 세트로 변환한다.

6.9 Skip-gram 모델 준비 작업

```
from torch import optim
```

```
device = 'mps' if torch.backends.mps.is_available() else "cpu"  
word2vec = VanillaSkipgram(vocab_size=len(token_to_id), embedding_dim=128)  
criterion = nn.CrossEntropyLoss().to(device)  
optimizer = optim.SGD(word2vec.parameters(), lr = 0.1)
```

- `VanillaSkipgram` 클래스의 단어 사전 크기(`vocab_size`)에 전체 단어 집합의 크기를 전달하고 임베딩 크기(`embedding_dim`)는 128로 할당
- 손실 함수는 단어 사전 크기만큼 클래스가 있는 분류 문제이므로 교차 엔트로피를 사용한다. 교차 엔트로피는 내부적으로 소프트 맥스 연산을 수행하므로 신경망 출력값을 후처리 없이 활용할 수 있다.

6.10 모델 학습

```
for epoch in range(10):  
    cost = 0.0  
    for input_ids, target_ids in dataloader:  
        input_ids = input_ids.to(device)  
        target_ids = target_ids.to(device)  
  
        logits = word2vec(input_ids)  
        loss = criterion(logits, target_ids)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
        cost += loss  
  
    cost = cost / len(dataloader)  
    print(f"Epoch: {epoch+1:4d}, Cost: {cost:.3f}")
```

- 모델 학습이 완료되면 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬 중 하나의 행렬을 선택해 임베딩 값을 추출한다.

```
Epoch: 1, Cost: 5.982
Epoch: 2, Cost: 5.933
Epoch: 3, Cost: 5.903
Epoch: 4, Cost: 5.881
Epoch: 5, Cost: 5.863
Epoch: 6, Cost: 5.848
Epoch: 7, Cost: 5.836
Epoch: 8, Cost: 5.824
Epoch: 9, Cost: 5.814
Epoch: 10, Cost: 5.804
```

6.11 임베딩 값 추출

- $W_{V \times E}$ 행렬에서 임베딩 값을 추출해보자

```
token_to_embedding = dict()
embedding_matrix = word2vec.embedding.weight.detach().cpu().numpy()

for word, embedding in zip(vocab, embedding_matrix):
    token_to_embedding[word] = embedding

index = 30
token = vocab[30]
token_embedding = token_to_embedding[token]
print(token)
print(token_embedding)
```

```
연기
[-1.9815925 -0.27938595 -0.66432756  0.15459386 -0.8807213  0.14786002
  0.64498925  0.1671892 -0.2414137  0.04376874 -0.7302386  0.18936345
 -0.57505965  1.0381693 -0.5814681  0.33503428  1.2105416  0.744385
 -0.8410904  0.61983615 -0.21655789  0.9868787 -0.83648425 -0.39607635
 -0.7772915  1.1710385  0.08705818 -1.0495017 -0.9618623 -0.86243856
 -0.9551921  0.43525833  0.4038037  0.0091143  0.6162521  0.16072837
 -0.8632577 -1.0972307  1.1155233  1.7636108 -1.3620842  1.2533647
  0.6578477 -0.7646224  2.4733486 -1.46763  0.6717474  0.6240405
 -0.59723425 -0.01888536  1.5286694  0.78765893  2.1047812 -0.53783995
  0.32375962  0.06312134  1.4547968 -0.3007019  0.17569126 -0.40104353
 -1.6516781  0.51341116 -0.20451538 -0.9581142  0.11553036 -0.15327851
 -0.03467102 -0.18076742  0.55899113 -1.0317096  1.0995387 -0.35254836
 -0.43045923  1.1635298  0.17001708 -1.7297022  0.8083983 -2.111588
  0.899763  0.9920264 -0.51245517 -0.9879903  0.9823478  0.72695404
  0.7069915  1.3645579  0.6577197  0.96688724  0.29578435  0.44688606
  0.01904913  0.82689625 -0.9165475 -0.27537212 -0.08054402 -1.0686257
  1.3339039  0.57266647  0.94790393 -1.0309244  0.62310606 -0.29960042
  0.34017724 -0.24073568 -0.1503162  0.71331966  1.6648158  1.6369427
 -1.0111179 -0.8155938  1.7836896 -1.0541843  0.57169664  0.23281881
 -0.813153 -0.56990284  0.3673754 -0.8648332 -0.34815615 -0.5190444
 -0.67202115  1.1868273 -0.01071477 -0.38852993  0.22591609  2.2127147
  0.986456  0.70623213]
```

- Word2Vec 모델의 임베딩 행렬을 이용해 각 단어의 임베딩 값을 매핑하고, 인덱스 30 값의 단어와 임베딩 값을 출력하여 단어 간의 유사도를 확인할 수 있다. 임베딩 유사도를 측정할 때 가장 일반적으로 **코사인 유사도(Cosine Similarity)**가 사용된다.
 - 코사인 유사도는 두 벡터 간의 각도를 이용하여 유사도를 계산하며, 두 벡터가 유사할 수록 값이 1에 가까워지고, 다를 수록 0에 가까워진다.
 - 코사인 유사도는 두 벡터의 내적을 벡터 크기의 곱으로 나눠 계산한다.

수식 6.10 코사인 유사도

$$\text{cosine similarity}(a,b) = \frac{a \cdot b}{\|a\| \|b\|}$$

- a와 b는 유사도를 계산하려는 벡터이며, 두 벡터를 내적한 벡터의 크기의 곱을 나눠 코사인 유사도를 계산할 수 있다. 벡터 크기는 각 성분의 제곱합에 루트를 씌운 값이다.

6.12 단어 임베딩 유사도 계산

```
import numpy as np
from numpy.linalg import norm # euclid norm

def cosine_similarity(a, b):
    cosine = np.dot(b, a) / (norm(b, axis = 1) * norm(a)) # 내적 / 두 벡터 크기의 곱
    return cosine

def top_n_index(cosine_matrix, n):
    closest_indexes = cosine_matrix.argsort()[::-1] # reverse
    top_n = closest_indexes[1: n+1]
    return top_n

cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n = 5)

print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도: {cosine_matrix[index]:.4f}")
```

연기와 가장 유사한 5 개 단어
연기력 - 유사도: 0.3214
여자 - 유사도: 0.3126
답답하다 - 유사도: 0.2931
황당한 - 유사도: 0.2860
고딩 - 유사도: 0.2720

- `cosine_similarity` 함수는 입력 단어와 단어 사전 내의 모든 단어와의 코사인 유사도를 계산한다.
 - `a` 매개변수는 임베딩 토큰, `b` 매개변수는 임베딩 행렬을 의미한다.
 - 임베딩 행렬은 [5001, 128]의 구조를 가지므로 노름을 계산할 때 $axis = 1$ 방향으로 계산한다.
- `top_n_index` 함수는 유사도 행렬을 내림차순으로 정렬해 어떤 단어가 가장 가까운 단어인지 반환한다. 입력 단어도 단어 사전에 포함되므로 두 번째 가까운 단어부터 계산해 반환한다.
- 입력된 단어가 '연기'일 때 가장 유사한 단어 3개로 '연기력', '여자', '답답하다'가 추출되었다.

모델 실습: Gensim

- 매우 간단한 구조의 Word2Vec 모델을 학습할 때 데이터 수가 적은 경우에도 학습하는데 오랜 시간이 소요된다. 계층적 소프트맥스나 네거티브 샘플링 같은 기법을 사용하면 더 효율적으로 학습할 수 있다.
- **젠심(Gensim) 라이브러리**
 - Word2Vec과 같은 자연어 처리 모델을 쉽게 구성할 수 있다.
 - 대용량 텍스트 데이터의 처리를 위한 메모리 효율적인 방법을 제공해 대규모 데이터셋에서도 효과적으로 모델을 학습할 수 있다.
 - 학습된 모델을 저장하여 관리할 수 있고, 비슷한 단어 찾기 등 유사도와 관련된 기능도 제공하여 자연어 처리에 필요한 다양한 기능을 제공한다.
- 젠심 라이브러리는 사이썬(Cython)을 이용해 병렬 처리나 네거티브 샘플링 등을 적용한다. 사이썬은 C++ 기반의 확장 모듈로 파이썬 모듈로 컴파일하는 기능을 제공한다. 젠심으로 모델을 구성하면 파이토치를 이용한 학습보다 훨씬 더 빠른 속도로 학습할 수 있다.
- 젠심 라이브러리의 Word2Vec 클래스는 네거티브 샘플링 등에 필요한 여러 하이퍼파라미터를 받아 쉽고 빠르게 모델을 학습할 수 있다.

Word2Vec 클래스


```
word2vec = gensim.models.Word2Vec(
    sentences=None,
    corpus_file=None,
    vector_size=100,
    alpha=0.025,
    window = 5 ,
    min_count=5,
    workers=3,
    sg = 0 ,
    hs = 0 ,
    cbow_mean=1,
    negative=5,
    ns_exponent=0.75,
    max_final_vocab=None,
    epochs=5,
    batch_words=10000)
```

- **sentences** : 모델의 학습 데이터 (토큰 리스트)
- **corpus_file** : 말뭉치 파일. 학습 데이터를 파일로 입력할 때 파일의 경로. 입력 문장 대신 사용 가능
- **vector_size** =100: 학습할 때 벡터의 크기를 의미하며, 임베딩 차원의 수를 설정한다.
- **alpha** : Word2Vec 모델의 학습률
- **window** : 학습 데이터를 생성할 윈도우의 크기. 윈도우가 3이라면 중심단어로부터 3만큼 떨어진 단어까지 주변 단어로 고려해 데이터를 생성한다.
- **min_count** : 학습에 사용할 단어의 최소 빈도. 말뭉치 내에 최소 빈도만큼 등장하지 않은 단어는 학습에 사용되지 않는다.
- **workers** : 빠른 학습을 위해 병렬 학습할 스레드의 수
- **sg** : Skip-gram 모델 사용 여부 (1: Skip-gram, 0: CBoW)
- **hs** : 계층적 소프트 맥스 사용 여부 (1: 사용, 0: 사용 x)
- **cbow_mean** : CBoW 평균화 (CBoW 모델로 구성할 때 사용). 중심 단어와 주변 단어를 합쳐서 하나의 벡터로 만들 때, 합한 벡터의 평균화 여부 설정 (1: 평균화, 0: 평균화 x)
- **negative** : 네거티브 샘플링의 단어 수 (0: 네거티브 샘플링 사용 x)
- **ns_exponent** : 네거티브 샘플링 확률 지수

- `max_final_vocab` : 단어 사전의 최대 크기. 최소 빈도를 충족하는 단어가 최대 최종 단어 사전보다 많으면 자주 등장한 단어 순으로 단어 사전을 구축한다.

6.13 Word2Vec 모델 학습

```
from gensim.models import Word2Vec

word2vec = Word2Vec(
    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=1,
    sg=1,
    epochs=3,
    max_final_vocab=10000
)
```

- 모델 학습 속도를 앞선 `VanillaSkipgram` 클래스와 비교해보면 매우 빠르게 학습되는 것을 확인할 수 있다. `word2vec` 인스턴스는 파이토치의 저장 메서드와 동일한 방법으로 저장할 수 있다.

6.14 임베딩 추출 및 유사도 계산

```
word = '연기'
print(word2vec.wv[word])
print(word2vec.wv.most_similar(word, topn = 5))
print(word2vec.wv.similarity(w1 = word, w2 = "연기력"))
```

```
[[-0.32991582 -0.3077436  0.1018536  0.3394808 -0.10078396  0.07406776
-0.08742826 -0.0783589 -0.5418248  0.5030264 -0.08281668 -0.31624553
 0.02647373 -0.00716113  0.23071702  0.06036479 -0.21342942  0.37393048
-0.18596834  0.16234246  0.684702  0.26185456 -0.24195373 -0.19425535
-0.18497355  0.08184844 -0.4752375  0.0449965  0.09078958 -0.19087994
-0.53809595  0.0183689  0.38310072 -0.07961736 -0.03978975 -0.02633742
 0.01659325 -0.07892772 -0.0017791 -0.13347657  0.02801502 -0.03882441
-0.38101333 -0.27940565 -0.27319863 -0.00205845 -0.39466995 -0.21730027
 0.12114257  0.2191869  0.42713597  0.35757875  0.23723166  0.14919747
-0.4523237 -0.0657582  0.08176541  0.27661917 -0.25127667  0.15919068
-0.00791723 -0.22857046  0.2043587  0.2297673 -0.21646923  0.1594185
-0.09707396  0.33163607  0.2369715 -0.29150516 -0.48337558 -0.27094063
-0.3725709  0.20694615  0.08468869 -0.28375745 -0.25209275 -0.22031595
-0.02853148  0.17922851 -0.25525835 -0.09100151  0.26517478  0.609612
 0.12871985  0.01571447  0.1286693 -0.5886281  0.03815779 -0.18360524
-0.34957418 -0.03001731 -0.0026088  0.07474415  0.09241483  0.16482718
-0.049757 -0.34113863 -0.07483633 -0.7847205 -0.3910966 -0.02837691
 0.19273056 -0.5760019  0.04031465  0.25778642  0.59107393 -0.01956522
 0.03148661 -0.30455962  0.4167551 -0.23062004 -0.16326569  0.3218601
 0.19463037 -0.39850748  0.4474581  0.5165976 -0.03531627  0.4621123
-0.25889298 -0.33451295  0.02277683  0.05976502  0.15538946 -0.03289662
-0.49381152 -0.13495162]
(['연기력', 0.7871235013008118), ('캐스팅', 0.7532762289047241), ('연기자', 0.7378984093666077), ('조연', 0.7256463766098022), ('몸매', 0.7188649773597717)]
0.78712344
```

- `word2vec` 인스턴스의 `wv` 속성은 학습된 단어 벡터 모델을 포함한 `Word2VecKeyedVectors` 객체를 반환한다. 이 객체는 단어 벡터 검색과 유사도 계산 등의 작업을 수행하는데 사용한다.
 - 주어진 단어에 대해 가장 유사한 단어를 찾는 `most_similar` 메서드와 두 단어 간의 유사도를 계산하는 `similarity` 메서드를 제공한다.

- Word2Vec은 분포 가설을 통해 쉽고 빠르게 단어의 임베딩을 학습할 수 있지만, 이는 단어의 형태학적 특징을 반영하지 못한다는 한계가 있다.
- 예를 들어 교착어로서 한국어는 어근과 접사, 조사 등으로 이루어지는 규칙을 가지고 있기 때문에 Word2Vec 모델에서는 이러한 구조적 특징을 제대로 학습하기가 어렵고, 이는 제한된 단어 사전에서 많은 OOV를 발생시키는 원인이 된다.

fastText

- 2015년 메타에서 개발한 오픈소스 임베딩 모델로, 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘이다. fastText는 단어와 문장을 벡터로 변환하는 기술을 기반으로 하며, 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는데 사용된다.
- Word2Vec과 유사하지만, fastText는 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능을 제공한다. 하위 단어를 고려하기 위해 N-gram을 사용해 단어를 분해하고 벡터화하는 방법으로 동작한다.
- fastText에서는 단어의 벡터화를 위해 `<`, `>`와 같은 특수 기호를 사용해 단어의 시작과 끝을 나타낸다. 이러한 기호는 단어의 하위 문자열을 고려하는데 중요한 역할을 한다.
- 기호가 추가된 단어는 N-gram을 사용해 **하위 단어 집합(Subword set)**으로 분해된다. '서울특별시'를 바이그램으로 분해하면 '서울', '울특', '특별', '별시'가 된다.
 - 분해된 하위 단어 집합에는 나뉘지지 않은 단어 자신도 포함되며, 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 갖게 된다.
 - 이러한 하위 단어 벡터들은 단어의 벡터 표현을 구성하며, 이를 사용해 자연어 처리 작업을 수행한다.

입력 단어	〈 〉 더하기	N-gram 분해	전체 단어 추가
서울특별시	〈서울특별시〉	〈서울 서울특 울특별 특별시 별시〉	〈서울 서울특 울특별 특별시 별시〉 〈서울특별시〉

그림 6.11 fastText의 하위 단어 집합

- fastText는 입력으로 받은 '서울특별시'라는 토큰에 대해 다음과 같은 단계를 거쳐 처리한다.
 1. 토큰의 양 끝에 '<와>'를 붙여 토큰의 시작과 끝을 인식할 수 있게 한다.
 2. 분해된 토큰은 N-gram을 사용해 하위 단어 집합으로 분해한다. 트라이그램을 사용한다면, '서울특별시'는 [<서울, 서울특, 율특별, 특별시, 별시>]로 분해된다.
 3. 분해된 하위 단어 집합에는 나뉘지지 않은 토큰 자체도 포함된다. 이렇게 하위 단어 집합이 만들어지면, 각 하위 단어는 고유한 벡터값을 갖는다.
- fastText는 위와 같은 방법으로 하위 단어 집합을 생성한다. 입력 단어를 하위 단어로 분해하면 총 6개의 하위 단어가 존재한다. 각 하위 단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터를 계산한다.
- 일반적으로 fastText는 다양한 N-gram을 적용해 입력 토큰을 분해하고 하위 단어 벡터를 구성함으로써 단어의 부분 문자열을 고려하는 유연하고 정확한 하위 단어 집합을 생성한다.
 - 즉, 같은 하위 단어를 공유하는 단어끼리는 정보를 공유해 학습할 수 있다. 이를 통해 비슷한 단어끼리는 비슷한 임베딩 벡터를 갖게 되어, 단어 간 유사도를 높일 수 있다.
 - 또한, OOV 단어도 하위 단어로 나누어 임베딩을 계산할 수 있게 된다.
 - 예를 들어 '개인택시, 정보처리 기사.' 임대차보호법'이라는 단어가 말뭉치 내에 있을 때, 이 단어들의 하위 단어들로부터 '개인정보보호법'이라는 단어의 임베딩도 연산할 수 있다.

모델 실습

- fastText와 Word2Vec 모델 모두 단어 임베딩을 학습하는데 사용되는 알고리즘이다. 두 알고리즘 모두 단어를 고정 길이 벡터 형태로 표현하기 위해 **분산 표현 학습**을 수행하고 **주변 단어의 정보를 활용하여 단어의 의미를 파악**한다. 이를 통해 단어 간 유사성을 측정하고, 비슷한 의미를 가진 단어를 유사한 벡터로 표현한다. 유사한 단어들은 공간상 가깝게 임베딩된다.
- fastText 모델도 CBoW와 Skip-gram으로 구성되며 네거티브 샘플링 기법을 사용해 학습한다. Word2Vec 모델은 기본 단위로 학습했다면, fastText는 하위 단어로 학습한다. 그러므로 Word2Vec 모델보다 입력 단어를 구성하는데 조금 더 복잡한 과정이 필요하고 모든 하위 단어 크기를 갖는 임베딩 계층이 필요하다.
- 젠심 라이브러리는 이러한 복잡한 과정을 FastText 클래스로 제공한다. Word2Vec과 기술적인 공통점이 많으므로 Word2Vec의 대부분 매개변수를 공유한다.

FastText 클래스

```

fasttext = gensim.models.FastText(
    sentences=None,
    corpus_file=None,
    vector_size=100,
    alpha=0.025,
    window=5,
    min_count=5,
    workers=3,
    sg=0,
    hs=0,
    cbow_mean=1,
    negative=5,
    ns_exponent=0.75,
    max_final_vocab=None,
    epochs=5,
    batch_words=10000,
    min_n=3,
    max_n=6
)

```

- 대부분은 Word2Vec 클래스의 하이퍼 파라미터와 동일하지만, N-gram 범위를 결정하는 하이퍼 파라미터가 추가되었다.
 - `min_n` : 사용할 N-gram의 최솟값
 - `max_n` : 사용할 N-gram의 최댓값

6.15 KorNLI 데이터세트 전처리

```

from Korpora import Korpora

corpus = Korpora.load("kornli")
corpus_text = corpus.get_all_texts() + corpus.get_all_pairs()
tokens = [sentence.split() for sentence in corpus_text]

print(tokens[:3])

```

```
[[ '개념적으로', '크림', '스키밍은', '제품과', '지리라는', '두', '가지', '기본', '차원을', '가지고',
'있다.', ], [ '시존', '중요', '알고', '있는', '거', '알아?', '네', '레벨에서', '다음', '레벨로',
'잃어버리는', '거야', '브레이브스가', '모험을', '떠올리기로', '결정하면', '브레이브스가', '트리플',
'A에서', '한', '남자를', '떠올리기로', '결정하면', '더블', 'A가', '그를', '대신하러', '올라가고',
'A', '한', '명이', '그를', '대신하러', '올라간다.', ], [ '우리', '번호', '중', '하나가', '당신의',
'지시를', '세밀하게', '수행할', '것이다.' ]]
```

- KorNLI(Korean Natural Language Inference) 데이터세트는 한국어 **자연어 추론(National Language Inference, NLI)**을 위한 데이터세트다. 자연어 추론이란 두 개 이상의 문장이 주어졌을 때, 두 문장 간의 관계를 분류하는 작업을 의미한다.
 - 이를 통해 주어진 문장이 **함의 관계(entailment)**, **중립 관계(neutral)**, **불일치 관계(contradiction)** 중 어느 관계에 해당되는지 분류할 수 있다.
 - 예를 들어 '나는 일본으로 여행을 가고 싶다'와 '파이토치를 공부하고 있다'라는 두 문장이 있다면 이 두 문장의 관계는 서로 연관이 없으므로 불일치 관계로 분류될 수 있다.
- 코포라 라이브러리의 KorNLI 인스턴스는 `get_all_texts` 와 `get_all_pairs` 메서드를 제공한다.
 - `get_all_texts` 메서드는 모든 문장을 튜플 형태로 반환하며,
 - `get_all_pairs` 메서드는 입력 문장과 쌍을 이루는 대응 문장을 튜플 형태로 반환한다.
- 단어 임베딩 모델을 학습하는 것이 목적이므로 두 문장의 관계를 고려하지 않고 입력 문장과 대응 문장 전체를 이용해 학습을 진행한다. Word2Vec 모델과 다르게, fastText 모델은 입력 단어의 구조적 특징을 학습할 수 있다. 따라서 형태소 분석기를 통해 토큰화하지 않고 띄어쓰기를 기준으로 단어를 토큰화해 학습을 진행한다.

6.16 fastText 모델 실습

- 전처리된 토큰을 생성했다면 젠심 라이브러리의 FastText 클래스를 활용해 모델을 학습한다.

```
from gensim.models import FastText

fastText = FastText(
    sentences=tokens,
    vector_size=128, # 임베딩 벡터 크기
    window = 5, # 윈도우 크기
    min_count = 5, # 5 번 이상 등장한 단어만 사용
    sg = 1,
    epochs = 3,
    min_n = 2, # N-gram: [2, 6]
```

```
max_n = 6
)
```

6.17 fastText OOV 처리

- Word2Vec 모델과는 다르게 OOV 단어를 대상으로도 의미 있는 임베딩을 추출할 수 있다.

```
oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn = 5))
```

- 단어 사전에 없는 단어의 임베딩을 추출하고 유사도를 계산한다.
- fastText 모델의 `wv.index_to_key` 메서드는 학습된 단어 사전을 나타내는 리스트를 의미한다. '사랑해요'라는 단어는 단어 사전 리스트 내에 존재하지 않으므로 OOV 토큰으로 처리된다.
- Word2Vec 모델은 단어 사전에 존재하지 않는 단어는 임베딩을 계산할 수 없지만, fastText는 하위 단어로 나뉘어 있기 때문에 '사랑해요'라는 단어를 처리할 수 있다.
 - fastText는 '사랑해요' 토큰을 '사랑', '랑해', '해요' 등의 하위 단어로 분해된하고, 분해된 단어의 임베딩을 모두 합해 전체 토큰의 임베딩을 계산하기 때문에 다른 단어에서 등장했던 '사랑'이나 '해요'와 같은 하위 단어를 통해 '사랑해요' 토큰의 임베딩을 계산할 수 있다.
 - 이와 같은 방식으로 fastText는 OOV 문제를 효과적으로 해결할 수 있다. 특히, 한국어와 같은 언어는 형태적 구조를 갖고 있기 때문에 하위 단어로 나누어 임베딩을 학습하는 fastText의 접근 방식을 이용해 OOV 문제를 해결할 수 있다.

순환 신경망 (RNN)

- 순환 신경망 (Recurrent Neural Network, RNN) 모델은 **순서가 있는 연속적인 데이터 (Sequence data)**를 처리하는데 적합한 구조를 갖고 있다. 순환 신경망은 각 시점 (Time Step)의 데이터가 이전 시점의 데이터와 **독립적이지 않다**는 특성 때문에 효과적으로 작동한다.
 - 예를 들어, 일일 주가 데이터세트가 있다고 가정한다면 3월 7일의 주가는 3월 6일 주가의 영향을 받았을 가능성이 높다. 동일하게 3월 6일의 주가는 3월 5일의 주가

의 영향을 받았을 가능성이 높다. 특정 시점 t 에서의 데이터가 이전 시점 $(t_0, t_1, \dots, t_{n-1})$ 의 영향을 받는 데이터를 연속형 데이터라한다.

- 자연어 데이터는 연속적 데이터의 일종으로 볼 수 있다. 자연어 데이터는 문장 안에서 단어들 **순서대로 등장**하므로, 각 단어는 **이전에 등장한 단어의 영향을 받아 해당 문장의 의미를 형성**한다. 예를 들어
 - 금요일이 지나면...
 - 수요일이 지나면 목요일이 되고, 목요일이 지나면 금요일, 금요일이 지나면...
- 이 문장들은 단어들의 연속으로 이루어진 데이터가 가지는 특징을 잘 보여준다. 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추할 수 있다. 첫 번째 문장을 보고 '주말' 또는 '토요일' 등의 단어가 등장할 것이라 예측할 수 있고, 두 번째 문장을 보고 '주말'보다는 '토요일'이 더 자연스러운 예측일 것이다.
- 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 상관관계가 존재한다. 자연어 처리에서는 문맥과 상호작용을 모델링해 정확한 의미 파악이 필요하다.

- **순환 신경망**은 연속적 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류로, 이전에 처리한 데이터를 다음 단계에 활용하고 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는데 사용된다.
 - 순환 신경망은 시계열 데이터, 자연어 처리, 음성 인식 및 기타 시퀀스 데이터와 같은 도메인에서 널리 사용된다. 이러한 데이터는 일반적으로 길이가 가변적이며, 순서에 따라 의미가 달라진다.
- 순환 신경망은 연속형 데이터를 순서대로 입력받아 처리하며 각 시점마다 **은닉 상태 (Hidden state)**의 형태로 저장한다. 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드를 **순환 신경망의 cell**이라한다.
 - 순환 신경망의 셀은 이전 시점의 은닉 상태 h_{t-1} 을 입력으로 받아 현재 시점의 은닉 상태 h_t 를 계산한다. 이러한 재귀적 특징으로 인해 '순환'신경망이라 불린다.

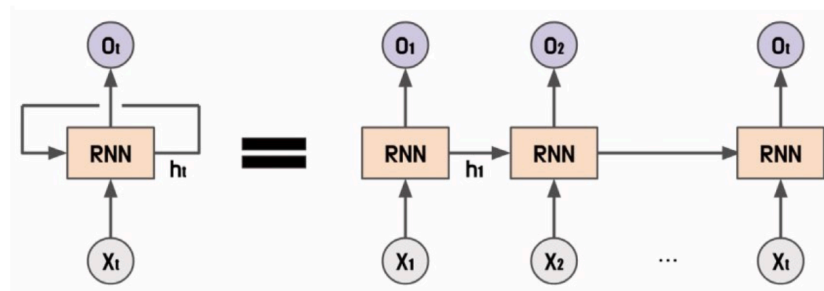


그림 6.12 순환 신경망의 셀

- 순환 신경망은 각 시험 t 에서 현재 입력값 x_t 와 $t - 1$ 시점의 은닉 상태 h_{t-1} 을 이용해 현 시점의 은닉 상태 h_t 와 출력값 y_t 를 계산한다.
- 은닉 상태의 수식은 다음과 같다.

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- σ_h 는 가중치와 편향을 이용해 계산한다.
- W_{hh} 는 이전 시점의 은닉 상태(h_{t-1})에 대한 가중치
- W_{xh} 는 입력값 x_t 에 대한 가중치
- b_h 는 은닉 상태 h_t 의 편향
- 출력값은 다음과 같이 계산한다.

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- σ_y 는 순환 신경망 출력값 계산을 위한 활성화 함수
- 출력값 활성화 함수는 현재 시점의 은닉 상태 h_t 를 입력으로 받아 출력값 y_t 를 계산한다.
- W_{hy} 는 현재 시점의 은닉 상태(h_t)에 대한 가중치
- b_y 는 출력값 y_t 의 편향
- 순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측하므로 연속형 데이터를 처리할 수 있다.

순환 신경망 구조

일대다 구조 (One-to-Many)

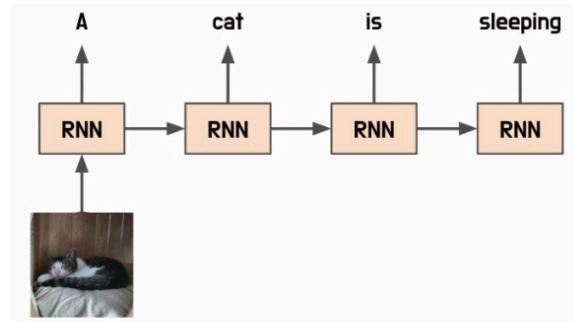


그림 6.13 순환 신경망의 일대다 구조

- 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조다.
 - 예를 들어, 자연어 처리 분야에서는 일대다 구조를 사용해 문장을 입력으로 받고, 문장에서 각 단어의 품사를 예측하는 작업을 할 수 있다. 입력 시퀀스는 문장으로 이루어져 있으며, 출력 시퀀스는 각 단어의 품사이다.
 - 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 **이미지 캡셔닝 (Image Captioning) 모델**이 된다. 입력 시퀀스는 이미지, 출력 시퀀스는 이미지에 대한 설명 문장들이다.
- 일대다 구조를 구현하기 위해서는 출력 시퀀스의 길이를 미리 알고 있어야 한다. 이를 위해 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이를 예측하는 모델을 함께 구현해야 한다.

다대일 구조(Many-to-One)

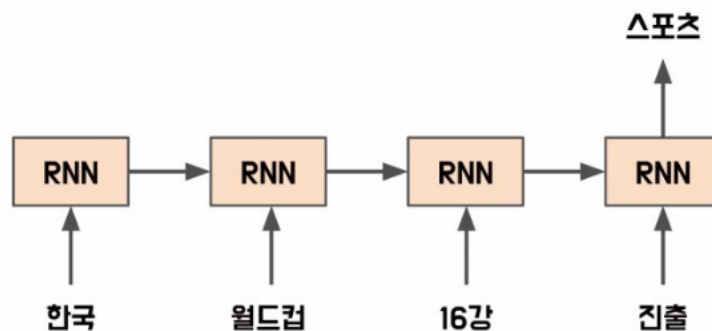


그림 6.14 순환 신경망의 다대일 구조

- 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조
 - 예를 들어 감성 분류(Sentiment Analysis) 분야에서는 다대일 구조를 사용해 특정 문장의 감정(긍정, 부정)을 예측하는 작업을 할 수 있다. 입력 시퀀스는 문장으로, 출력값은 해당 문장의 감정(긍정, 부정)으로 이루어져 있다.

- 입력 시퀀스가 어떤 범주에 속하는지 구분하는 문장 분류, 두 문장 간의 관계를 추론하는 **자연어 추론(Natural Language Inference)** 등에도 적용할 수 있다.

다대다구조 (Many-to-Many)

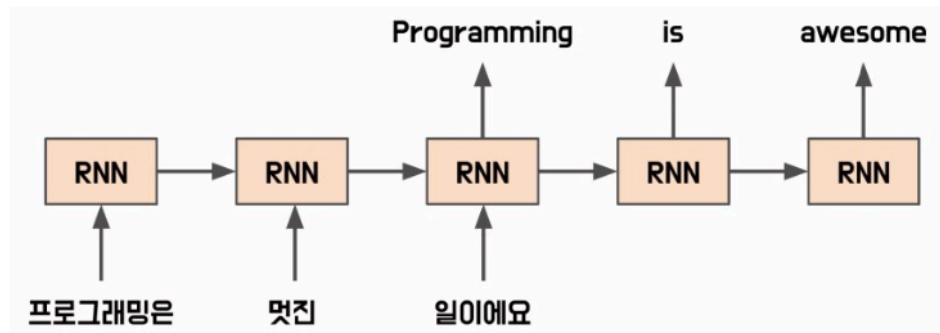


그림 6.15 순환 신경망의 다대다 구조

- 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조
 - 입력 문장에 대해 번역된 출력 문장을 생성하는 번역기
 - 음성 시스템에서 음성 신호를 입력으로 받아 문장을 출력하는 음성 인식기
- 다대다 구조에서 입력 시퀀스와 출력 시퀀스의 길이가 서로 다른 경우가 있을 수 있다. 입력과 출력 시퀀스 길이를 맞추기 위해 패딩을 추가하거나 잘라내는 등의 전처리 과정이 수행된다.
- 다대다 구조는 **시퀀스-시퀀스(Seq2Seq)** 구조로 이뤄져 있다. 시퀀스-시퀀스 구조는 입력 시퀀스를 처리하는 인코더(Encoder)와 출력 시퀀스를 생성하는 디코더(Decoder)로 구성된다.
 - 인코더: 입력 시퀀스를 처리해 고정 크기의 벡터 출력
 - 디코더: 이 벡터를 입력으로 받아 출력 시퀀스 생성

양방향 순환 신경망 (Bidirectional Recurrent Neural Network, BiRNN)

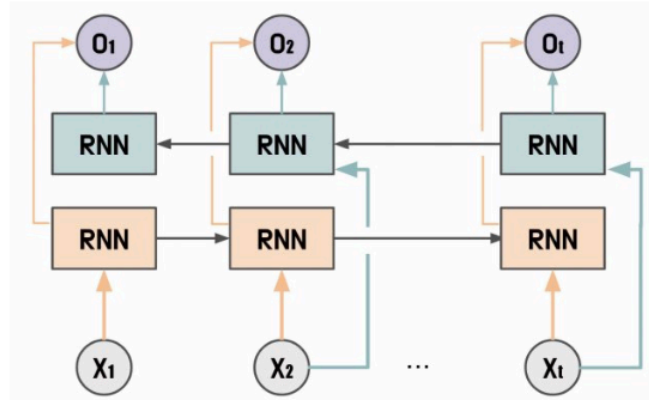


그림 6.16 양방향 순환 신경망 구조

- 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식
- 순환 신경망은 현재 시점의 입력값을 처리하기 위해 이전 시점($t - 1$)의 은닉 상태를 이용하는데, 양방향 순환 신경망에서는 이전 시점의 은닉 상태 뿐만 아니라 이후 시점($t + 1$)의 은닉 상태도 함께 이용한다.
 - “인생은 B와 _ 사이의 C다”라는 문장에서 _에 입력될 단어를 예측해야한다면 앞의 문장만이 아니라 뒤의 문장에도 영향을 받는다는 것을 알 수 있다.
- 양방향 순환 신경망은 t 시점 이후의 데이터도 t 시점의 데이터를 예측하는데 사용될 수 있다. 이러한 방법은 입력 데이터를 순방향으로 처리하는 것만 아니라 역방향으로 거꾸로 읽어들이어 처리하는 방식으로 이루어진다.
- 대부분 연속형 데이터는 이전 시점 데이터뿐만 아니라 이후 시점 데이터와 큰 상관관계를 갖고 있으므로 양방향 순환 신경망은 양방향적인 정보를 모두 고려하여 현재 시점 출력값을 계산한다.

다중 순환 신경망 (Stacked Recurrent Neural Network)

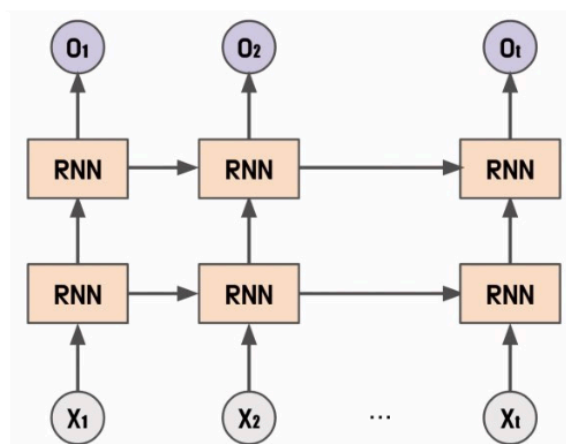


그림 6.17 다중 순환 신경망 구조

- 여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환 신경망이 서로 다른 정보를 처리하도록 설계되어 있다.
- 여러 개의 순환 신경망 층으로 구성되며, 각 층에서의 출력값은 다음 층으로 전달되어 처리된다. 여러 개 층으로 구성되어 데이터의 다양한 특징을 추출할 수 있어서 성능이 향상될 수 있고, 층이 깊어질 수록 더 복잡한 패턴을 학습할 수 있다는 장점이 있다.
- 하지만 순환 신경망 층이 많아질 수록 학습 시간이 오래 걸리고, 기울기 소실 문제가 발생할 가능성도 높아진다.

순환 신경망 클래스

```
rnn = torch.nn.RNN(
    input_size,
    hidden_size,
    num_layers=1,
    nonlinearity= "tanh",
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False
)
```

- 입력 특성 크기 (`input_size`) 와 은닉 상태 크기 (`hidden_size`)를 입력해 순환 신경망을 구성할 수 있다. 입력 특성은 입력값 x 를 의미하며 , 은닉 상태는 h 를 의미한다.
- 계층 수 (`num_layers`): 순환 신경망 층수 (2 이상이면 다중 순환 신경망)
- 활성화 함수 (`nonlinearity`): 순환 신경망에서 사용되는 활성화 함수 σ 의 종류 (`tanh` or `relu`)
- 편향 (`bias`): 편향값 사용 유무
- 배치 우선 (`batch_first`): 입력 배치 크기를 첫 번째 차원으로 설정할지 여부
 - 참: [배치 크기 , 시퀀스 길이, 입력 특성 크기]
 - 거짓: [시퀀스 길이 , 배치 크기 , 입력 특성 크기]
- 드롭아웃 (`dropout`): 과적합 방지를 위한 드롭아웃 확률
- 양방향 순환 신경망 (`bidirectional`): 순환 신경망 구조가 양방향으로 처리할지 여부

6.18 양방향 다층 신경망

```

import torch
from torch import nn

input_size = 128
output_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size=input_size,
    hidden_size=output_size,
    num_layers=num_layers,
    nonlinearity='tanh',
    batch_first=True,
    bidirectional=bidirectional,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size)

outputs, hidden = model(inputs, h_0)
print(outputs.shape)
print(hidden.shape)

```

```

torch.Size([4, 6, 512])
torch.Size([6, 4, 256])

```

- 순환 신경망 클래스는 순전파 연산시 입력값(`inputs`)과 초기 은닉 상태(`h_0`)로 순방향 연산을 수행해 출력값(`outputs`)과 최종 은닉 상태(`hidden`)를 반환한다.
- 입력값 차원은 [배치 크기 , 시퀀스 길이, 입력 특성 크기] (`batch_first = True`)로 전달한다.
 - 초기 은닉 상태는 순방향 계층 수와 양방향 구조에 따라 전달해야하는 크기가 달라진다. [계층 수 x 양방향 여부 + 1, 배치 크기, 은닉 상태 크기]로 구성된다. 이 예제

에서는 $[3 \times 2, 4, 256]$ 형태로 전달된다.

- 출력값은 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) x 은닉 상태 크기]가 되므로 $[4, 6, 2 \times 256]$ 형태로 반환된다.
- 최종 은닉 상태는 초기 은닉 상태와 동일한 차원으로 반환되므로 $[3 \times 2, 4, 256]$ 형태로 반환된다.
- 순환 신경망 입출력 차원은 배치 우선 매개 변수에 따라 차원의 형태가 달라지므로 매개 변수 설정에 주의한다.

장단기 메모리 (Long Short-Term Memory, LSTM)

- LSTM은 1997년 셉 호흐라이터와 유르겐 슈미트후버가 제안한 알고리즘으로 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 모델이다.
- 일반적 순환 신경망은 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값을 예측하는 구조이므로 시간적으로 먼 과거의 정보는 잘 기억하지 못한다. 하지만 어떤 연속형 데이터는 다음 시점의 데이터를 유추하기 위해 훨씬 먼 시점의 데이터에서 힌트를 얻어야 한다.
 - Ex. 우리는 시내에 있는 중식당에 갔다. 식당의 분위기는 괜찮았고 식사는 맛있었다. 그녀와의 대화도 즐거웠다. 우리는 함께 대화를 더 나누기 위해 _을 떠나 카페로 이동했다. 라는 문장이 있다.
 - 이 빈칸을 채우기 위해서는 먼 과거에 등장한 정보를 기억하고 유추해야 한다.
- 순환 신경망은 시간적으로 연속된 데이터를 다룰 수 있지만, 앞선 시점에서의 정보를 끊임없이 반영하기 때문에 학습 데이터의 크기가 커질수록 앞서 학습한 정보가 충분히 전달되지 않는다는 단점이 있다.
- 이는 **장기 의존성 문제 (Long-term dependencies)**를 발생시킬 수 있고, 활성화 함수 (하이퍼볼릭 탄젠트 함수/ReLU 함수) 특성으로 인해 역전파 과정에서 **기울기 소실**이나 **기울기 폭주 문제**를 발생시킬 수 있다.
- 이러한 문제를 해결하기 위해 장단기 메모리를 사용한다. 장단기 메모리는 순환 신경망과 비슷한 구조를 가지지만, **메모리 셀(Memory Cell)**과 **게이트(Gate)**라는 구조를 도입해 순환 신경망의 문제를 완화한다.
- 장단기 메모리 구조

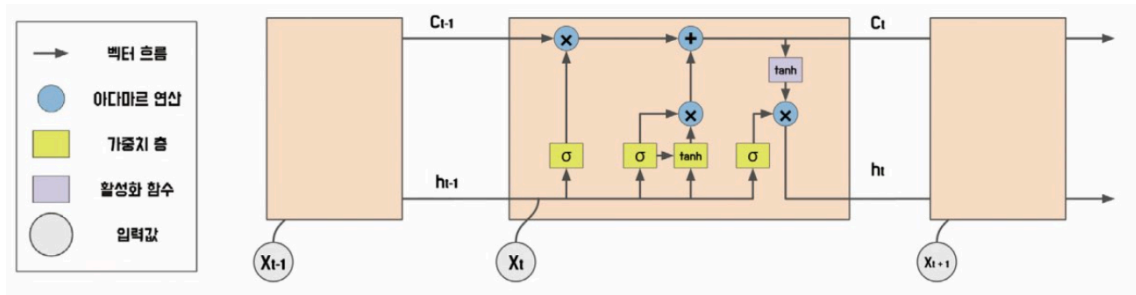


그림 6.18 장단기 메모리 구조

- 장단기 메모리는 **셀 상태, 망각 게이트, 기억 게이트, 출력 게이트**로 정보의 흐름을 제어한다.
 - 셀 상태**: 정보를 저장하고 유지하는 메모리 역할 (출력 게이트와 망각 게이트에 의해 제어됨)
 - 망각 게이트**: 장단기 메모리에서 이전 셀 상태에서 어떤 정보를 삭제할지 결정. 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 삭제할지 결정
 - 입력 게이트**: 새로운 정보를 어떤 부분에 추가할지 결정하는 역할. 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 추가할지 결정
 - 출력 게이트**: 셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할. 현재 입력과 이전 셀 상태, 그리고 새로 추가된 정보를 입력으로 받아 출력할 정보를 결정

장단기 메모리 구조

- 장단기 메모리 신경망 내에서 셀 상태의 연산 과정은 다음과 같다.



메모리셀 → 망각 게이트 → 기억 게이트 → 출력 게이트

- 이러한 게이트들은 입력값에 대한 가중치를 동적으로 조절하면서, 적절한 정보만을 기억하고 유지할 수 있다는 장점을 가지고 있다.

메모리셀

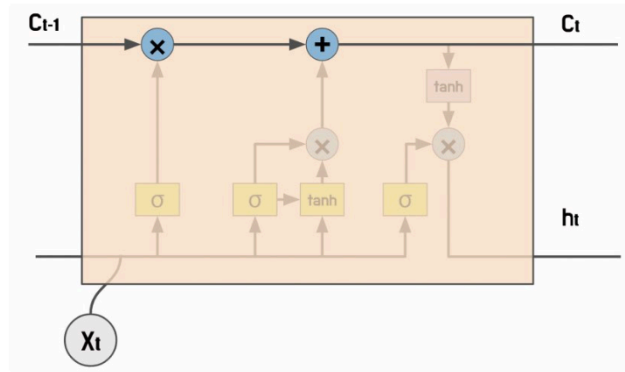


그림 6.19 장단기 메모리 셀 상태

- **메모리셀**은 순환 신경망의 은닉 상태와 유사하게 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장하는 역할을 한다.
 - 하지만 순환 신경망에서 은닉 상태는 출력값을 계산하는데 사용되지만, **메모리셀은 출력값 계산에 직접 사용되지 않는다**. 대신 장단기 메모리 구조는 망각 게이트, 입력 게이트, 출력 게이트를 통해 어떤 정보를 버릴지, 어떤 정보를 기억할지, 어떤 정보를 출력할지를 결정하는 추가적인 연산을 수행한다.
- 망각 게이트, 입력 게이트, 출력 게이트 세 게이트 모두 시그모이드 활성화 함수를 사용한다. 시그모이드 함수는 입력값을 0과 1 사이 값으로 변환하므로 게이트의 출력값은 각각 0에서 1 사이의 값으로 결정된다. 이 값이 해당 게이트가 입력값에 대해 얼마나 많은 정보를 통과시킬지 결정한다.
 - 망각 게이트의 출력값이 1이면 이전 시점의 기억 상태가 현재 시점의 기억 상태에 완전히 유지되며, 0이면 이전 시점의 기억 상태는 현재 시점의 기억 상태에 전혀 반영되지 않는다.

망각 게이트

망각 게이트는 *현재 시점의 입력과 이전 시점의 은닉 상태*를 입력으로 받아 시그모이드 함수를 거친 값과 메모리 셀을 곱한 값으로 **이전 시점의 메모리 셀을 업데이트**한다. 시그모이드 함수를 거치면 0과 1 사이의 값이 출력되며, 이 값은 어떤 정보를 유지할 것인지 아니면 망각할 것인지를 결정한다.

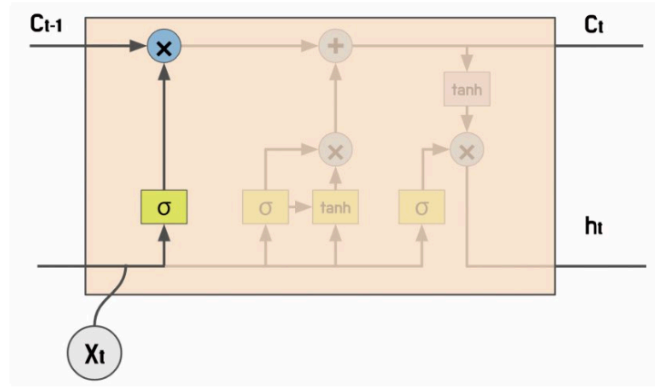


그림 6.20 장단기 메모리의 망각 게이트

- 망각 게이트는 이전 시점의 은닉 상태 h_{t-1} 과 현재 시점의 입력값 x_t 를 통해 연산을 수행한다.

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

- 수식에서 $W_x^{(f)}$ 와 $W_h^{(f)}$ 는 입력값과 은닉 상태를 위한 가중치를 의미하며, $b^{(f)}$ 는 망각 게이트의 편향이다.
- 현재 시점의 입력값 x_t 와 $t - 1$ 시점의 은닉 상태 h_{t-1} 을 사용해 망각 게이트의 출력값을 계산하게 된다.
- 망각 게이트는 두 가중치를 통해 망각 게이트 출력값을 최적화한다. 망각 게이트 출력값은 메모리 셀을 계산하기 위한 가중치로 사용된다.
- 망각 게이트는 이전 시점의 메모리 셀과 현재 시점의 입력을 바탕으로 **어떤 정보를 삭제** 할지 결정한다.
- 망각 게이트의 출력값은 0 에서 1 사이의 값으로, 이 값이 1 에 가까우면 더 많은 정보를 유지하고 반대로 0 에 가까우면 더 많은 정보를 삭제한다. 출력값이 정확히 1 이라면 아무런 정보를 삭제하지 않으며, 0 이라면 모든 정보를 삭제한다.
- 메모리 셀을 계산하기 위해 기억 게이트 출력값을 더해 최종 메모리 셀 값을 계산한다.

기억 게이트

두 번째 연산은 **기억 게이트**로 **현재 시점의 입력과 이전 시점의 은닉 상태**를 입력으로 받아 시그모이드 함수를 거친 값과 하이퍼볼릭 탄젠트 함수를 거친 값의 곱으로 **새로운 기억 값을 계산**한다.

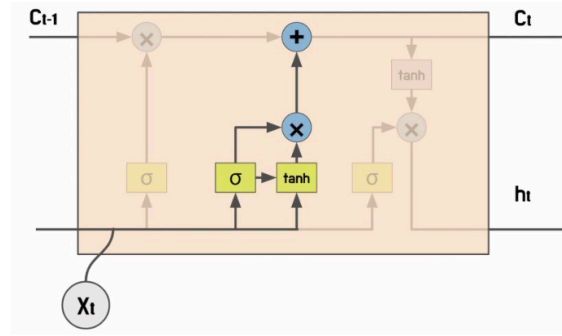


그림 6.21 장단기 메모리의 기억 게이트

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_t = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

- 기억 게이트는 g_i 와 i_t 로 구성돼 있으며 망각 게이트와 동일한 연산으로 계산된다. g_i 는 활성화 함수로 하이퍼볼릭 탄젠트를 사용하며, i_t 는 시그모이드를 사용한다.
 - 시그모이드 함수(g_i)는 입력값의 중요도를 결정하고, 하이퍼볼릭 탄젠트 함수는 입력값을 -1과 1 사이의 값으로 변환한다. 하지만 g_i 만으로는 새로운 은닉 상태를 계산하기 위해 이 은닉 상태를 얼마나 기억할지 제어하기 어렵기 때문에 i_t 값으로 새로운 은닉 상태의 기억을 제어한다.
 - i_t 는 [0, 1]의 값을 가지므로 현재 시점에서 얼마나 많은 정보를 기억할 것인지를 결정하는 가중치 역할을 한다. i_t 가 1에 가까울 수록 기억할 정보가 많아지고, 0에 가까울수록 정보를 망각하게 된다.
- 메모리 셀은 망각 게이트와 기억 게이트의 정보로 현재 시점의 메모리 셀 값을 계산한다. 최종 메모리셀 계산 방법은 다음과 같다.

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

- f_t 는 망각 게이트 출력값, c_{t-1} 은 이전 시점의 메모리 셀 값이다. 이 값을 원소별 곱셈 연산 (아다마르 곱)하면 현재 시점의 메모리 셀 값이 계산된다.
- 망각 게이트 값이 0에 가까울 수록 이전 시점의 메모리 셀 값이 현재 시점의 메모리 셀 값에 영향을 미치지 않게 된다.
- 기억 게이트도 아다마르 곱을 통해 계산되며 망각 게이트와 합산된다. 망각 게이트는 이전 시점의 메모리 셀을 얼마나 유지할지 결정하며, 기억 게이트는 현재 시점의 새로운 정보를 얼마나 받아들일지를 결정한다.

- 메모리 셀이 계산되었다면 어떤 정보를 출력할지 출력 게이트에서 결정한다.

출력 게이트

세 번째 연산은 **출력 게이트**로 **현재 시점의 입력과 이전 시점의 은닉 상태**, 그리고 새로 업데이트된 메모리셀을 입력으로 받아 **현재 시점의 출력값을 계산**한다. 출력 게이트는 어떤 정보를 출력할지 결정한다.

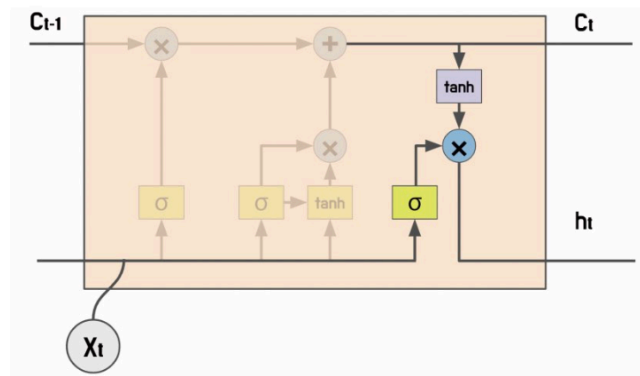


그림 6.22 장단기 메모리의 출력 게이트

- 출력 게이트는 현재 시점의 은닉 상태를 제어한다. 출력 게이트도 망각 게이트에서 사용된 수식과 동일한 수식을 사용하며, 활성화 함수도 시그모이드를 사용한다.

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)} x_t + W_h^{(o)} h_{t-1} + b^{(o)})$$

- 시그모이드 함수는 입력값을 [0, 1] 범위로 변환하므로 현재 시점의 은닉 상태 값을 얼마나 사용할지 결정한다. 출력 게이트와 현재 메모리 셀을 연산하면 새로운 은닉 상태를 계산할 수 있다.

수식 6.17 은닉 상태 갱신

$$h_t = o_t \odot \tanh(c_t)$$

- 현재 시점의 은닉 상태 h_t 는 출력 게이트와 하이퍼볼릭 탄젠트를 적용한 메모리 셀 값으로 계산된다. 출력 게이트는 [0, 1]의 범위를 가지며, 하이퍼볼릭 탄젠트가 적용된 메모리 셀은 [-1, 1] 범위를 갖는다. 두 값을 아다마르 곱 연산하면 현재 시점의 은닉 상태가 이전 시점의 은닉 상태에서 얼마나 영향을 받는지 계산할 수 있게 된다.

장단기 메모리 클래스

```
lstm = torch.nn.LSTM(
    input_size,
    hidden_size,
    num_layers=1,
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False,
    proj_size = 0
)
```

- 장단기 메모리 클래스는 RNN과 거의 유사한 구조를 갖는다. 장단기 메모리 클래스는 활성화 함수를 명확하게 정의해 사용하므로 순환 신경망에서 사용하는 활성화 함수 (`nonlinearity`) 매개 변수를 사용하지 않는다.
- 투사 크기 (`proj_size`) 매개 변수를 사용해 장단기 메모리 계층의 출력에 대한 선형 투사 (Linear Projection) 크기를 결정한다.
 - 투사 크기가 0보다 큰 경우, 은닉 상태를 선형 투사를 통해 다른 차원으로 매핑한다. 이 값을 통해 출력 차원을 줄이거나 다른 차원으로 변환할 수 있다.
 - 투사 크기가 0이라면 은닉 상태의 차원을 변환하지 않고 그대로 유지한다.
- 장단기 메모리 클래스는 출력 차원을 조절할 때 투사 크기 매개 변수로 모델 크기와 복잡도를 변경할 수 있다. 그러므로 투사 크기(`proj_size`)는 은닉 상태 크기 (`hidden_size`)보다 작은 값으로 설정한다.

6.19 양방향 다층 장단기 메모리

```
import torch
from torch import nn

input_size = 128
output_size = 256
num_layers = 3
bidirectional = True
proj_size = 64

model = nn.LSTM(
    input_size = input_size,
    hidden_size = output_size,
```

```

num_layers=num_layers,
batch_first=True,
bidirectional = bidirectional,
proj_size = proj_size,
)

batch_size = 4
sequence_len = 6
inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(
    num_layers * (int(bidirectional) + 1),
    batch_size,
    proj_size if proj_size > 0 else output_size,
)

c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size)

outputs, (h_n, c_n) = model(inputs, (h_0, c_0))

print(outputs.shape)
print(h_n.shape)
print(c_n.shape)

```

```

torch.Size([4, 6, 128])
torch.Size([6, 4, 64])
torch.Size([6, 4, 256])

```

- LSTM은 순전파 연산시 입력값과 초기 은닉 상태, 초기 메모리 셀 상태(`c_0`)로 순방향 연산을 수행해 출력값(`outputs`)과 최종 은닉 상태(`h_n`), 최종 메모리셀 상태(`c_n`)를 반환한다.
- 입력값 차원은 앞선 순환 신경망 클래스에서 사용하던 입력값 차원 형태와 동일한 형태로 [배치 크기, 시퀀스 길이, 입력 특성 크기]를 전달한다.
- 초기 은닉 상태도 RNN과 동일한 구조를 갖지만, LSTM에서는 선형 투사를 통해 출력층 차원을 변경할 수 있으므로 투사 크기가 0보다 큰 경우 출력층의 차원을 투사 크기로 입력한다.
- 초기 메모리 셀은 순환 신경망 클래스의 초기 은닉 상태와 동일한 구조인 [계층 수 x 양방향 여부 + 1, 배치 크기, 은닉 상태 크기]를 갖는다. 초기 메모리 셀은 선형 투사를 적

용하더라도 은닉 상태 크기를 사용한다.

- 순방향 연산 결과는 출력값과 최종 은닉 상태, 최종 메모리 셀 상태가 반환된다. 출력 값은 수식도 순환 신경망과 동일한 수식을 사용하지만, 선형 투사가 적용되었으므로 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) x 투사 크기 (또는 은닉 상태 크기)] 로 구성된다. 제와 같은 구조라면 [4, 6, 2X64] 의 형태로 반환된다.
- 최종 은닉 상태는 초기 은닉 상태와 동일한 차원인 [3x2, 4, 256] 형태로 반환된다. 순환 신경망 입출력 차원은 배치 우선 매개 변수에 따라 차원 형태가 달라지므로 주의한다.
- LSTM은 순환 신경망과 비슷한 구조를 갖지만 , 투사 크기가 적용되는 경우 차원 계산 방식이 달라진다. 또한 메모리 셀의 상태도 입력으로 전달해야 하며 , 은닉 상태와 메모리 셀 상태를 튜플로 묶어사용하므로 사용에 주의한다.

모델 실습 (RNN, LSTM)

- 문장 긍정/부정 분류 모델 학습

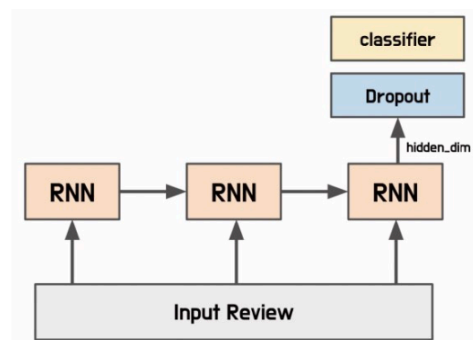


그림 6.23 긍정/부정 분류 모델 구조

- 임베딩은 임베딩 벡터를 가져오는 임베딩 계층을 의미한다. 임베딩 계층은 [어휘 사전의 크기 x 임베딩 벡터의 크기]로 순람표 구조를 갖는다.
임베딩 계층은 입력 텍스트를 정수 인코딩한 뒤 해당 색인 값에 해당하는 임베딩을 가져오는 역할을 한다. 초깃값으로는 무작위 값을 할당하고, 학습을 통해 임베딩 계층이 최적화되게 만들거나, 사전 학습된 임베딩 벡터를 가져와 사용할 수 있다.
- 이번 예제에서는 두 가지 방법을 모두 다뤄본다.

6.20 문장 분류 모델

```
from torch import nn
class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab, # 단어 사전 크기
```

```

        hidden_dim,
        embedding_dim,
        n_layers,
        dropout = 0.5,
        bidirectional = True,
        model_type = 'lstm' # 모델 종류에 따라 RNN, LSTM 사용 여부 결정
    ):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim = embedding_dim,
            padding_idx=0
        )
        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional = bidirectional,
                dropout=dropout,
                batch_first=True,
            )
        elif model_type == "lstm":
            self.model = nn.LSTM(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

        if bidirectional:
            # 양방향으로 분류기 계층을 구성하면 전달되는 입력 채널 수가 달라지므로 분류기 계층을 2개 구성
            self.classifier = nn.Linear(hidden_dim * 2, 1)
        else:
            self.classifier = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(dropout)

```



```
def forward(self, inputs):
    embeddings = self.embedding(inputs)
    # 입력 받은 정수 인코딩을 임베딩 계층에 통과시켜 임베딩 값을 얻음

    output,_ = self.model(embeddings)
    last_output = output[:, -1, :] # 출력값의 마지막 시점만 활용
    last_output = self.dropout(last_output)
    logits = self.classifier(last_output)
    return logits
```

6.21 데이터세트 불러오기

```
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac = 0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size:", len(train))
print("Testing Data Size:", len(test))
```

	text	label
33553	모든 편견을 날려 버리는 ...	1
9427	무한 리메이크의 소재. 감...	0
199	신날 것 없는 애니.	0
12447	잔잔 격동	1
39489	오랜만에 찾은 주말의 명...	1

Training Data Size : 45000
Testing Data Size : 5000

- `corpus.test` 로 테스트 세트를 불러오고, 이를 학습과 테스트 데이터로 분리

6.22 데이트 토큰화 및 단어 사전 구축

```
from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens= ['. ', '<pad>', '<unk>'])
# 문장 길이를 맞추기 위해 <pad> 토큰을 special_tokens에 추가
# 2 개의 특수 토큰과 단어 사전 최대 길이 5000개 -> 단어 사전 50002개

token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))
```

```
['<pad>', '<unk>', '.', '이', '영화', '의', '..', '가', '에', '...']
5002
```

6.23 정수 인코딩 및 패딩

```
import numpy as np

def pad_sequences(sequences, max_length, pad_value):
    result = list()
    for sequence in sequences:
```

```

sequence = sequence[: max_length]
pad_length = max_length - len(sequence)
padded_sequence = sequence + [pad_value] * pad_length
result.append(padded_sequence)
return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in train_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in test_tokens
]

max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])

```

```

[ 223 1716   10 4036 2095  193  755    4    2 2330 1031  220   26   13
 4839    1    1    1    2    0    0    0    0    0    0    0    0    0
    0    0    0    0]
[3307    5 1997  456    8    1 1013 3906    5    1    1   13  223   51
    3    1 4684    6    0    0    0    0    0    0    0    0    0
    0    0    0    0]

```

- 학습 데이터셋과 테스트 데이터셋을 단어 사전에 있는 정수로 인코딩한다. **정수로 인코딩하면 모델이 텍스트 데이터를 처리하기 쉬워진다.**
- 최대 길이 (`max_length`) 는 입력 텍스트의 길이를 고려해 결정한다. 너무 큰 값으로 설정하면 입력 행렬의 크기가 커져 많은 리소스를 필요로 하게 되며, 너무 짧은 경우 문장을 제대로 반영하지 못해 모델의 성능이 저하될 수 있다.
- `pad_sequences` 함수는 너무 긴 문장은 최대 길이로 줄이고 , 너무 작은 길이라면 최대 길이와 동일한 크기로 변환한다. 그러므로 `pad_sequences` 함수는 시퀀스를 최대 길이로 잘

라내고, 시퀀스 길이가 작으면 `<pad>` 토큰을 시퀀스 뒤에 이어 붙여 동일한 길이로 변경한다.

6.24 데이터 로더 적용

```
import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

- 텐서 데이터셋 (`TensorDataset`) 클래스는 파이토치 텐서 형태를 입력값으로 받는다. 따라서 정수 인코딩과 라벨값을 파이토치 텐서 형태로 변환하고 변환된 데이터 셋을 데이터 로더 클래스에 적용한다.

6.25 손실 함수와 최적화 함수 정의

```
from torch import optim

n_vocab = len(token_to_id)

hidden_dim = 64 # 은닉 상태 크기 64
embedding_dim = 128 # 임베딩 벡터 크기 128
n_layers = 2 # 신경망 2 개 층

device = "mps" if torch.backends.mps.is_available() else "cpu"

classifier = SentenceClassifier(
    n_vocab=n_vocab,
    hidden_dim=hidden_dim,
    embedding_dim = embedding_dim,
```

```

n_layers=n_layers
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

```

- 현재 모델은 문장의 긍 / 부정을 분류하므로 손실 함수는 이진 교차 엔트로피 함수를 적용한다.
 - `BCEwithLogitsLoss` 클래스는 `BCELoss` 클래스와 `Sigmoid` 클래스가 결합된 형태다. `BCELoss` 클래스를 사용하는 경우, `criterion(torch.sigmoid(output), target)` 과 동일한 형태로 간주할 수 있다.
- 최적화 함수는 **RMSProp(Root Mean Square Propagation)**을 적용한다. RMSProp 은 모든 기울기를 **누적하지 않고**, 지수 가중 이동 평균(Exponentially Weighted Moving Average, EWMA)을 사용해 학습률을 조절한다.
 - 즉, 기울기 제공 값의 평균값이 작아지면 학습률을 증가시키고, 그 반대일 경우 학습률을 감소시켜서 불필요한 지역 최솟값에 빠지는 것을 방지한다. RMSprop 은 기울기의 크기가 큰 경우에는 빠른 수렴을 보이며, 작은 경우에는 더 작은 학습률을 유지함으로써 더 안정적으로 최적화를 수행할 수 있다.

6.26 모델 학습 및 테스트

```

import numpy as np
import torch

# 학습 함수
def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()

```

```

optimizer.step()

if step % interval == 0:
    print(f"Train Loss {step} : {np.mean(losses)}")

# 평가 함수
def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        yhat = torch.sigmoid(logits) > 0.5
        corrects.extend(torch.eq(yhat, labels).cpu().tolist())

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(corrects)}")

# 전체 학습 루프
epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```

```

Train Loss 0 : 0.6886653900146484
Train Loss 500 : 0.6930192142665506
Train Loss 1000 : 0.6814309212115857
Train Loss 1500 : 0.6566959291914953
Train Loss 2000 : 0.6289551559759342
Train Loss 2500 : 0.6042968219027716
Val Loss : 0.4880547617761472, Val Accuracy : 0.7622

```

...

```

Train Loss 0 : 0.18164722621440887
Train Loss 500 : 0.2978266854352223
Train Loss 1000 : 0.30000228573615617
Train Loss 1500 : 0.3047617496047752
Train Loss 2000 : 0.3086865940212697
Train Loss 2500 : 0.3089462954689507
Val Loss : 0.39236691967843057, Val Accuracy : 0.8236

```

- 에포크마다 테스트 데이터셋으로 모델의 검증 손실과 검증 정확도를 확인한다. 이로써 모델이 새로운 데이터를 얼마나 잘 예측하는지 평가한다.
- 테스트 데이터셋에 대해서도 손실이 감소하며, 정확도가 상승한다.

6.27 학습된 모델로부터 임베딩 추출

- 모델 학습 과정에서 임베딩 계층을 비롯한 순환 신경망 내의 여러 가중치가 최적화된다. 학습된 임베딩 계층의 가중치를 동일한 단어 사전을 $\hookrightarrow$ 사용하는 토큰의 임베딩 값으로도 사용할 수 있다.

```

token_to_embedding = dict()
embedding_matrix = classifier.embedding.weight.detach().cpu().numpy()

for word, emb in zip(vocab, embedding_matrix):
    token_to_embedding[word] = emb

token = vocab [1000]
print(token, token_to_embedding[token])

```

```

보고싶다 [ 0.8253137  0.04844039  0.41631582  0.75111073  1.8635908  0.91580963
...
-0.01363746 -0.7661964 ]

```

- 학습된 모델의 임베딩 계층을 각 단어에 대한 임베딩으로 사용할 수 있지만, 긍/부정 분류는 임베딩 계층이 아닌 순환 신경망의 연산이 더 중요하게 동작한다. 모델이 복잡할 수록 임베딩 계층이 토큰의 의미 정보를 학습하기는 더 어렵다.

6.28 사전 학습된 모델로 임베딩 계층 초기화

- 사전 학습된 임베딩 값을 초깃값으로 적용해 모델을 학습시킨다.

```
from gensim.models import Word2Vec
word2vec = Word2Vec.load('models/word2vec.model')
init_embeddings = np. zeros ((n_vocab, embedding_dim))

for index, token in id_to_token. items():
    if token not in ["<pad>", "<unk>"]:
        init_embeddings[index] = word2vec.wv[token]

embedding_layer = nn. Embedding.from_pretrained(
    torch. tensor (init_embeddings, dtype=torch.float32)
)
```

- 사전 학습된 모델로 임베딩 계층을 초기화 하는 경우 넘파이 배열로 초기화 할 수 있다. 단, 이때 `special_tokens (<pad>, <unk>)` 은 제외한다.

6.29 사전 학습된 임베딩 계층 적용

```
from torch import nn
class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab, # 단어 사전 크기
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout = 0.5,
        bidirectional = True,
        model_type = 'lstm', # 모델 종류에 따라 RNN, LSTM 사용 여부 결정
        pretrained_embedding=None
    ):
        super().__init__()
        if pretrained_embedding is not None:
```



```

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32),
            freeze=False # 필요 시 True로 설정하여 학습 중 임베딩 고정 가능
        )
    else:
        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )

    if model_type == "rnn":
        self.model = nn.RNN(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            bidirectional = bidirectional,
            dropout=dropout,
            batch_first=True,
        )
    elif model_type == "lstm":
        self.model = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=n_layers,
            bidirectional=bidirectional,
            dropout=dropout,
            batch_first=True,
        )

    if bidirectional:
        # 양방향으로 분류기 계층을 구성하면 전달되는 입력 채널 수가 달라지므로 분류기 계층을 다르게 구성
        self.classifier = nn.Linear(hidden_dim * 2, 1)
    else:
        self.classifier = nn.Linear(hidden_dim, 1)
    self.dropout = nn.Dropout(dropout)

    def forward(self, inputs):

```

```

embeddings = self.embedding(inputs)
# 입력 받은 정수 인코딩을 임베딩 계층에 통과시켜 임베딩 값을 얻음
output,_ = self.model(embeddings)
last_output = output[:, -1, :] # 출력값의 마지막 시점만 활용
last_output = self.dropout(last_output)
logits = self.classifier(last_output)
return logits

```

- 사전 학습된 임베딩 (pretrained_embedding) 매개변수가 None 이 아니라면 전달된 값을 임베딩 계층으로 초기화한다.

6.30 사전 학습된 임베딩을 사용한 모델 학습

```

classifier = SentenceClassifier(
    n_vocab=n_vocab,
    hidden_dim=hidden_dim,
    embedding_dim = embedding_dim,
    n_layers=n_layers
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```

```
Train Loss 0 : 0.6927101612091064
Train Loss 500 : 0.5729601959625404
Train Loss 1000 : 0.5223068734714678
Train Loss 1500 : 0.501177434779103
Train Loss 2000 : 0.4927794478018245
Train Loss 2500 : 0.4885529308784299
Val Loss : 0.44360038442924, Val Accuracy : 0.7974
```

...

```
Train Loss 0 : 0.37388283014297485
Train Loss 500 : 0.35009346521662144
Train Loss 1000 : 0.3479555794498423
Train Loss 1500 : 0.3496556199744572
Train Loss 2000 : 0.35449253517514406
Train Loss 2500 : 0.35453654036241644
Val Loss : 0.3737754999353482, Val Accuracy : 0.8308
```

- 사전 학습된 임베딩을 사용하는 것은 모델 성능을 개선할 수 있는 방법 중 하나다. 하지만 학습 데이터의 양이 충분히 많다면, 모델의 목적에 맞게 새로운 임베딩 층을 학습하는 것이 더 좋은 결과를 얻을 수도 있다.
- 또한, 사전 학습된 임베딩을 사용하더라도 해당 언어와 문제에 맞는 임베딩을 선택하는 것이 중요하다. 예를 들어, 한국어 자연어 처리에서는 문맥 정보를 고려한 임베딩 방법이 더 성능이 좋을 수 있다. 따라서 모델의 목적과 데이터의 특성을 고려하여 적절한 임베딩 방법을 선택하는 것이 중요하다.

합성곱 신경망 (CNN)

- CNN은 주로 이미지 인식과 같은 컴퓨터 비전 분야의 데이터를 분석하기 위해 사용되는 인공 신경망이다. 합성곱 신경망은 입력 데이터의 지역적 특징을 추출하는데 특화된 구조를 갖고 있으며 이를 위해 합성곱 연산을 사용한다.
- 합성곱 연산은 이미지의 특정 영역에서 입력값의 분포 또는 변화량을 계산해 출력 노드를 생성한다. 특정 영역 안에서 연산을 수행하므로 **지역 특징 (Local Features)**을 효과적으로 추출할 수 있다.
- 이미지 데이터는 고정된 프레임 내에 객체들의 위치와 형태가 자유분방하므로 여러 영역의 지역 특징을 조합해 입력 데이터의 전반적인 **전역 특징 (Global Features)**을 파악할 수 있다.

합성곱 계층

- 합성곱 계층은 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층이다. 합성곱 계층은 이미지나 음성 데이터와 같은 고차원 데이터를 처리하는 데 주로 사용한다.
- 합성곱 계층은 필터를 사용해 데이터의 특징을 추출하므로 데이터의 지역적인 패턴을 인식할 수 있으며, 입력 데이터의 모든 위치에서 동일한 필터를 사용하므로 모델 매개변수를 공유한다.
 - 모델의 매개변수를 공유함으로써 모델이 학습해야 할 매개변수 수가 감소해 과대적합을 방지한다. 또한 입력 데이터에서 특징을 추출할 때, 해당 특징이 이미지 내 다른 위치에 존재하더라도 필터를 사용해 특징을 추출하므로 특징이 어디에 있어도 동일하게 추출할 수 있다.
 - 이러한 합성곱 계층을 여러 겹 쌓아 모델을 구성하며, 합성곱 계층이 많아질수록 모델의 복잡도가 증가하므로 더 다양한 특징을 추출해 학습할 수 있다.

필터

- 합성곱 계층은 입력 데이터에 필터(Filter)를 이용해 합성곱 연산을 수행하는 계층이다. 커널 또는 윈도우라고도 불리기도 하고, 일반적으로 정방향으로 구성된다. (3 × 3, 5 × 5)
- 이 필터를 일정 간격으로 이동하면서 입력 데이터와 합성 연산을 수행해 특징 맵을 생성한다. 필터 영역마다 합성곱 연산이 수행되며, 필터의 가중치가 모델 학습 과정에서 갱신된다.
- 합성곱 신경망은 보통 여러 개의 필터를 사용해 다양한 특징을 추출한다.

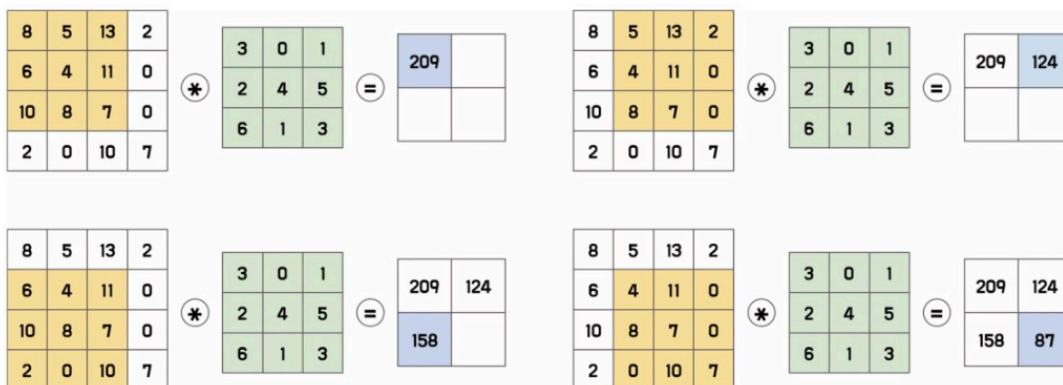


그림 6.24 특징 맵 추출 과정

- 4×4 이미지에 3×3 크기의 필터가 1 간격만큼 이동하면서 2×2 특징맵 추출
- 입력값은 이미지의 픽셀값을 의미하며, 필터의 값은 학습을 통해 변경되는 가중치로 임의의 초깃값을 설정

- 합성곱 연산은 입력 이미지에서 필터와 대응하는 부분을 곱한 후, 모두 더한 값을 특징 맵의 한 원소로 사용한다.
- 이러한 특징 맵은 다음 합성곱 계층의 입력으로 사용되어 동일한 합성곱 연산 과정을 반복한다. 각 계층에서 하나의 필터가 여러 번 사용되고 이를 공유 가중치로 공유함으로써 이미지 내에서 어느 위치에서도 동일한 패턴을 학습할 수 있게 된다.

패딩

- 일반적으로 합성곱 연산을 수행하면 출력값인 특징 맵의 크기가 작아진다. 즉, 입력값의 크기가 4X 4 이었지만, 2X2 크기로 감소한다.
- 이러한 현상을 방지하기 위해 입력 이미지나 입력으로 사용되는 특징 맵 가장자리에 특정 값을 덧붙이는 **패딩(Padding)**을 추가한다. 가장자리에 덧붙이는 패딩 값은 0 으로 할당하는데, 이를 **제로 패딩 (Zero padding)**이라고 한다.

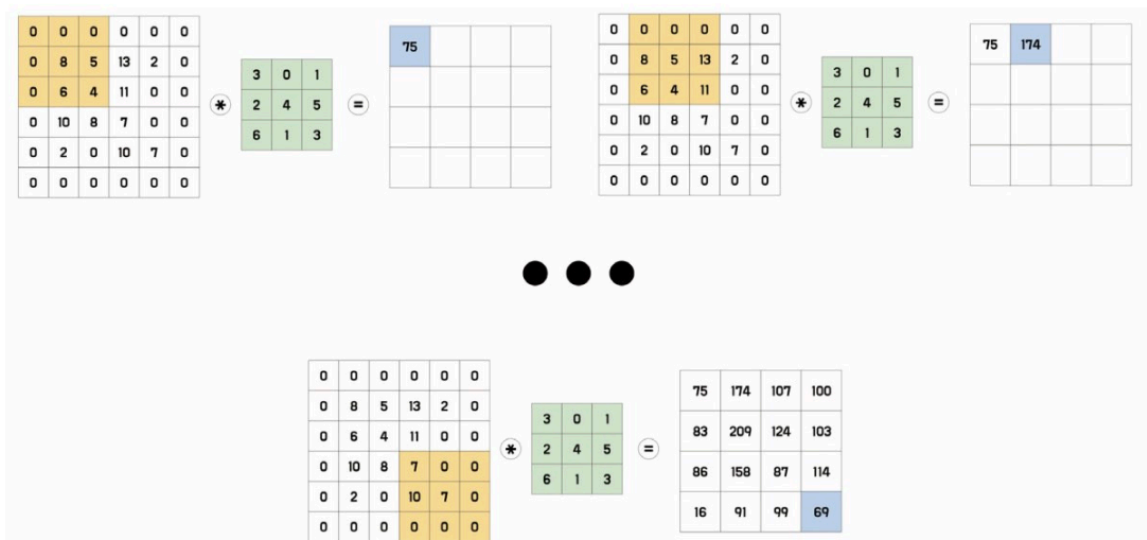


그림 6.25 제로 패딩

- 4 × 4 이미지 → 3 x 3 필터 → 4 x4 크기의 특징 맵

간격 (Stride)

- 필터가 한 번에 움직이는 크기. 간격의 크기가 1 이면 필터가 한 픽셀씩 이동하면서 합성곱 연산을 수행하며, 간격의 크기가 2 이상이면 필터가 더 큰 간격으로 이동하면서 계산을 수행한다.
- 간격의 크기를 조절함으로써 출력 데이터의 크기를 조절할 수 있다. 즉, 간격을 조정함으로써 입력 데이터의 공간적인 정보를 유지하거나 감소시킬 수 있다.

- 간격을 조절해 합성곱 신경망이 학습해야 하는 모델 매개변수의 수를 감소시킬 수 있으며, 이를 통해 모델의 복잡도를 낮추고 과대적합을 방지할 수 있다.

채널 (Channel)

- 입력 데이터와 필터 간의 연산은 채널(Channel) 에서 수행된다. 채널은 입력 데이터와 필터가 3 차원으로 구성되어 있을 때 같은 위치의 값끼리 연산되게 한다. 이를 통해 입력 데이터의 공간 정보를 유지하면서 추출되는 특징을 확장할 수 있다.
- 예를 들어, 입력 데이터가 RGB 이미지라면, 각각의 R, G, B 채널마다 동일한 필터가 존재하며, 각 필터는 해당 채널의 정보를 추출해 2D 맵을 생성한다.
- 채널 개수는 일반적으로 합성곱 계층에서 설정되며, 이는 모델의 구조나 목적에 따라 달라진다. 채널의 개수가 많아질수록 학습할 수 있는 특징의 다양성이 증가해 **모델의 표현력 (Representational Power) 이 높아지는 효과**를 가져온다.
 - 출력 채널이 많은 경우, 각 채널은 입력 데이터에서 서로 다른 특징을 학습할 수 있다. 따라서 모델은 더 많은 종류의 특징을 학습하게 되며, 그로 인해 더 복잡한 문제를 해결할 수 있는 능력을 갖추게 된다.
 - 그러나 출력 채널이 많을수록 모델의 매개변수가 많아지므로 학습 시간과 메모리 사용량이 증가하는 단점을 갖게 된다. 그러므로 모델이 과대적합 되는 위험이 발생한다.

팽창 (Dilation)

- 합성곱 연산을 수행할 때 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법으로 필터와 입력 데이터 사이에 간격을 두는 방법이다.
- 일반적으로 합성곱 계층에서 팽창값은 1로 사용해 간격을 한 칸만 띄워 사용한다. 이 값이 커질수록 필터가 입력 데이터를 바라보는 범위가 넓어진다.

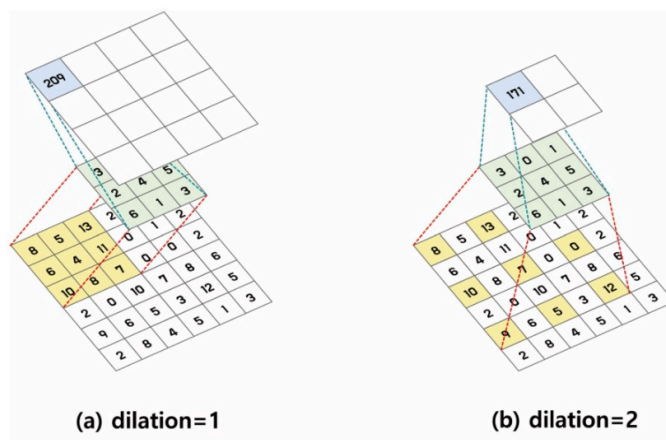


그림 6.26 팽창

- 6×6 크기의 이미지에 3×3 크기의 필터를 1 간격만큼 이동하고 패딩이 0 일 때 특징 맵을 추출하는 과정을 시각화한 것
- 팽창은 필터의 크기를 키우지 않고 입력 데이터에 더 넓은 영역을 고려할 수 있게 해 더 깊고 복잡한 모델을 구성할 수 있게 한다. 팽창을 적절히 적용하면 모델의 매개변수 수를 줄일 수 있어 메모리 사용량을 줄일 수 있다.
- 하지만 필터가 바라봐야 하는 입력 데이터의 범위가 커지므로 오히려 연산량이 늘어날 수도 있다. 또한 팽창 크기가 너무 크다면 인접한 픽셀값을 고려하지 않게 되므로 공간적인 정보가 보존되지 않아 특징 추출의 효과가 떨어질 수 있다.

합성곱 계층 클래스

```
conv = torch.nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size,
    stride=1,
    padding=0,
    dilation=1,
    groups= 1,
    bias=True,
    padding_mode="zeros")
```

- 그룹 (**groups**)은 입력 채널과 출력 채널을 하나의 그룹으로 묶는 것.
 - 그룹을 2 이상의 값으로 설정하면 입력 채널과 출력 채널을 더 작은 그룹으로 나눠 각각 합성곱 연산을 수행한다. 그룹을 사용하면 합성곱 연산 시 모델 매개 변수 수를 줄 일 수 있다.
 - 그룹의 값이 2 이상이면 입력 채널과 출력 채널의 수가 그룹 수로 나누어떨어져야 한다. 즉. 입력 채널이나 출력 채널이 그룹 수의 배수여야 한다.
- 합성곱 계층의 출력 크기는 다음과 같다.

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

- 값이 소수로 나오면 버림하여 정수로 반환한다.

활성화 맵 (Activation Map)

- 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값
- 합성곱 계층에서 입력 이미지와 필터의 합성곱 연산을 통해 특징 맵을 추출하면 이 값에 비선형성을 추가하기 위해 활성화 함수를 적용 (ReLU를 일반적으로 사용)
- 이렇게 얻은 활성화 맵은 다음 계층의 입력값으로 사용된다. 다음 계층이 합성곱 계층이라면 동일한 방식의 연산이 수행돼 합성곱 연산과 활성화 함수를 거치게 된다.

풀링 (Pooling)

- 특징 맵의 크기를 줄이는 연산으로 합성곱 계층 다음에 적용한다. 풀링은 특징 맵의 크기를 줄여 연산량을 감소시키고 입력 데이터의 정보를 압축하는 효과를 가진다.
- 크게 **최댓값 풀링 (Max Pooling)** 과 **평균값 풀링 (Average Pooling)**이 있다.

풀링 클래스

최대 풀링 클래스

```
pool = torch.nn.MaxPool2d(  
    kernel_size,  
    stride=None,  
    padding=0,  
    dilation = 1  
)
```

평균값 풀링 클래스

```
pool = torch.nn.AvgPool2d(  
    kernel_size,  
    stride=None,  
    padding=0,  
    dilation = 1  
)
```

- 2D 평균값 풀링 클래스의 출력 크기는 다음과 같다.

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

완전 연결 계층 (Fully Connected Layer)

- 각 입력 노드가 모든 출력 노드와 연결된 상태이기 때문에 입력과 출력 간의 모든 가능한 관계를 학습할 수 있다.
- 일반적으로 합성곱 신경망에서는 합성곱 계층과 풀링 계층을 거친 결과물인 특징 맵을 입력으로 받는다. 입력된 3 차원 특징 맵은 평탄화 작업을 통해 1 차원 벡터로 변경하고 완전 연결 계층의 가중치와 내적 연산을 수행해 출력값을 계산한다.
- 완전 연결 계층은 이전 계층에서 추출된 특징을 활용하여 최종적인 분류 작업을 수행하기 때문에 합성곱 신경망의 최종 출력을 결정하는 역할을 한다.

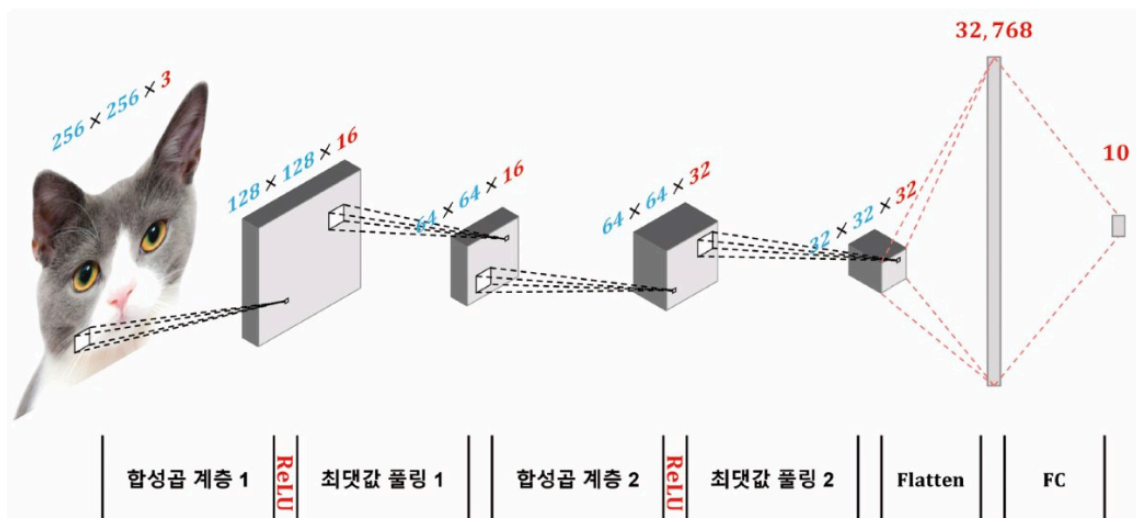


그림 6.28 합성곱 모델 구조 시각화

6.31 합성곱 모델

```
import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
```

```

super().__init__()

self.conv1 = nn.Sequential(
    nn.Conv2d(
        in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
    ),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
)

self.conv2 = nn.Sequential(
    nn.Conv2d(
        in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1
    ),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
)

# 완전 연결 계층은 선형 변환, 입력 벡터 x 가중치 곱으로 계산
self.fc = nn.Linear(32 * 32 * 32, 10)

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    x = torch.flatten(x)
    x = self.fc(x)
    return x

```

모델 실습

- 합성곱 신경망을 활용한 자연어 처리 모델을 구성해 본다.
- 2D 이미지와 달리 텍스트 데이터의 임베딩 값은 입력 순서를 제외하면 입력값의 위치가 의미를 가지지 않는다. 그러므로 텍스트 데이터에서도 이미지 데이터와 같이 2 차원 합성곱 필터를 사용하면 텍스트의 정보를 제대로 학습할 수 없으므로 1차원 합성곱을 적용해야 한다.

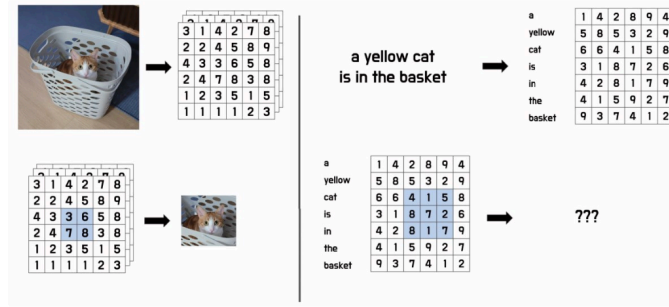


그림 6.29 이미지 데이터와 텍스트 데이터의 2차원 합성곱

- 1차원 필터의 크기는 필터의 높이에만 영향을 미치며, 필터의 너비는 입력 임베딩의 크기가 된다.

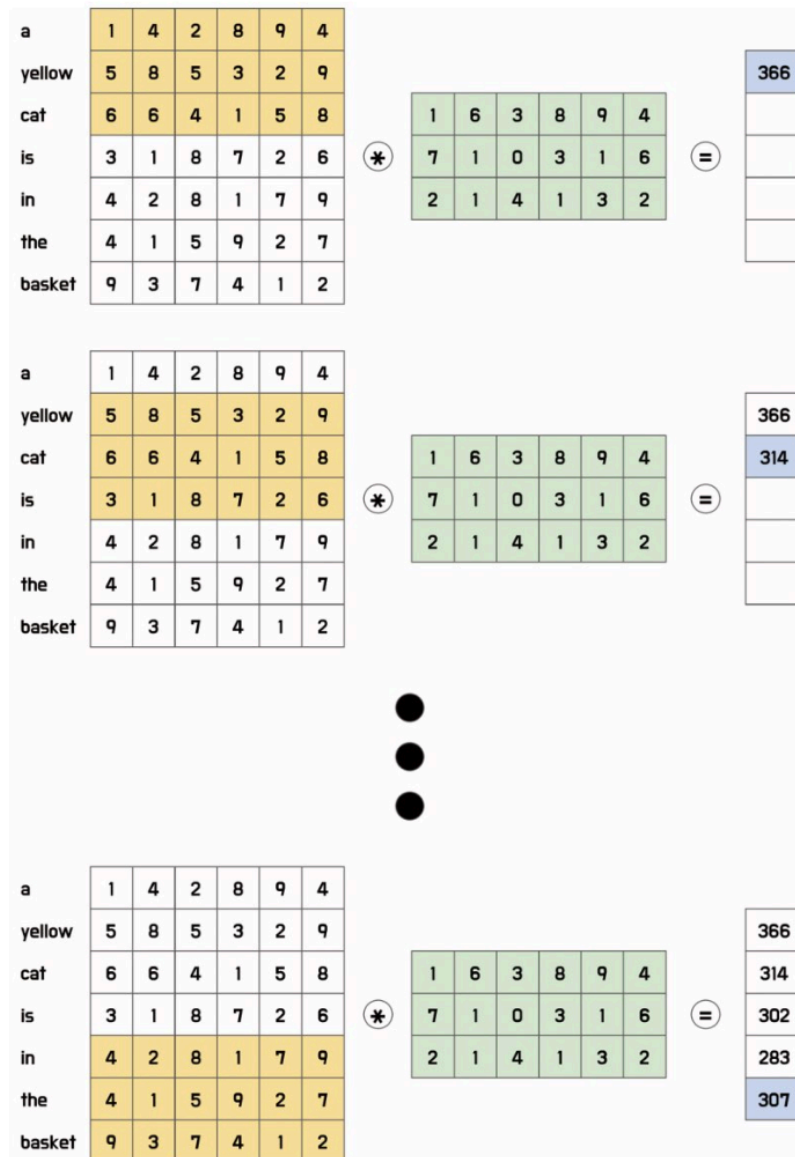


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

- 필터 크기가 3 인 합성곱 임베딩을 수행하면 인접한 3 개의 토큰에 대해 연산을 수행한다. 이는 앞서 다룬 N-gram 과 유사한 개념으로, 텍스트 순서에 따른 정보를 학습할 수 있다.
- 1 차원 합성곱을 이용한 신경망에서는 다양한 크기의 합성곱 필터를 사용하여 여러 종류의 정보를 추출할 수 있다. 1 차원 합성곱을 수행하면 1 차원 벡터를 출력값으로 얻게 되므로 풀링을 적용하면 하나의 스칼라값이 도출된다. 그러므로 크기가 다른 여러 개의 합성곱 필터를 사용하면 여러 개의 스칼라값을 얻을 수 있고 이러한 다양한 크기의 출력 벡터를 모아 하나의 벡터로 연결하여 하나의 특징 벡터로 만들 수 있다.

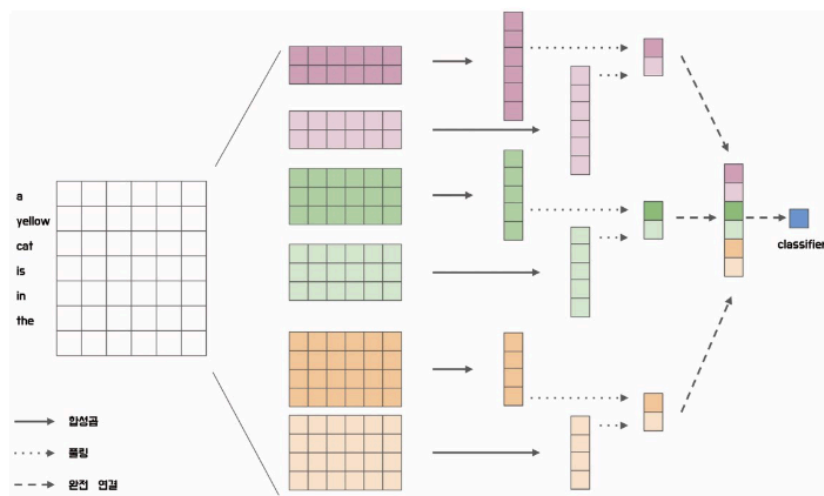


그림 6.31 1차원 합성곱을 이용한 출력 벡터 계산

6.32 합성곱 기반 문장 분류 모델 정의

```
from torch import nn
import torch

class SentenceClassifier(nn.Module):
    def __init__(self, pretrained_embedding, filter_sizes, max_length, dropout=0.5):
        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32)
        )
        embedding_dim = self.embedding.weight.shape[1]

        conv = []
        for size in filter_sizes:
```

```

conv.append(
    nn.Sequential(
        nn.Conv1d(
            in_channels=embedding_dim,
            out_channels=1,
            kernel_size=size
        ),
        nn.ReLU(),
        nn.MaxPool1d(kernel_size=max_length - size + 1),
    )
)
self.conv_filters = nn.ModuleList(conv)

output_size = len(filter_sizes)
self.pre_classifier = nn.Linear(output_size, output_size)
self.dropout = nn.Dropout(dropout)
self.classifier = nn.Linear(output_size, 1)

def forward(self, inputs):
    embeddings = self.embedding(inputs)
    embeddings = embeddings.permute(0, 2, 1)

    conv_outputs = [conv(embeddings) for conv in self.conv_filters]
    concat_outputs = torch.cat([conv.squeeze(-1) for conv in conv_outputs], 1)

    logits = self.pre_classifier(concat_outputs)
    logits = self.dropout(logits)
    logits = self.classifier(logits)
    return logits

```

- `SentenceClassifier` 클래스는 사전 학습된 임베딩 벡터 (`pretrained_embedding`), 필터 크기 리스트 (`filter_sizes`), 최대 길이 (`max_length`) 를 입력받아 모델을 구성한다.
- 순방향 메서드에서는 입력값 (`inputs`)을 임베딩 계층에 통과시켜 임베딩 벡터를 얻는다. 이후 `permute` 메서드를 통해 차원을 변경하고 앞서 설정한 여러 개의 합성곱 필터에 통과시킨다. `cat` 메서드를 통해 각 필터의 출력값을 이어 붙여 하나의 벡터로 만들고 분류를 위한 네트워크를 통과시켜 최종 출력값을 얻는다.

6.33 합성곱 신경망 분류 모델 학습

```

filter_sizes = [3, 3, 4, 4, 5, 5]

classifier = SentenceClassifier(
    pretrained_embedding=init_embeddings,
    filter_sizes=filter_sizes,
    max_length=max_length
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

```

```

Train Loss 0 : 0.6765573620796204
Train Loss 500 : 0.5889187179758639
...
Train Loss 2000 : 0.4638506230534702
Train Loss 2500 : 0.4630201855560438
Val Loss : 0.44208170073672226, Val Accuracy : 0.798

```

- 필터 크기는 그림 6.31 과 동일한 구조인 [3, 3 , 4, 4, 5, 5] 의 구조를 입력한다. 필터 크기 변수의 값과 개수를 조절해 모델의 구조를 변경하거나 확장할 수 있다.
- 최적화 함수는 순환 신경망에서 사용한 RMSProp 이 아닌 **Adam(Adaptive Moment Estimation)**을 사용한다.